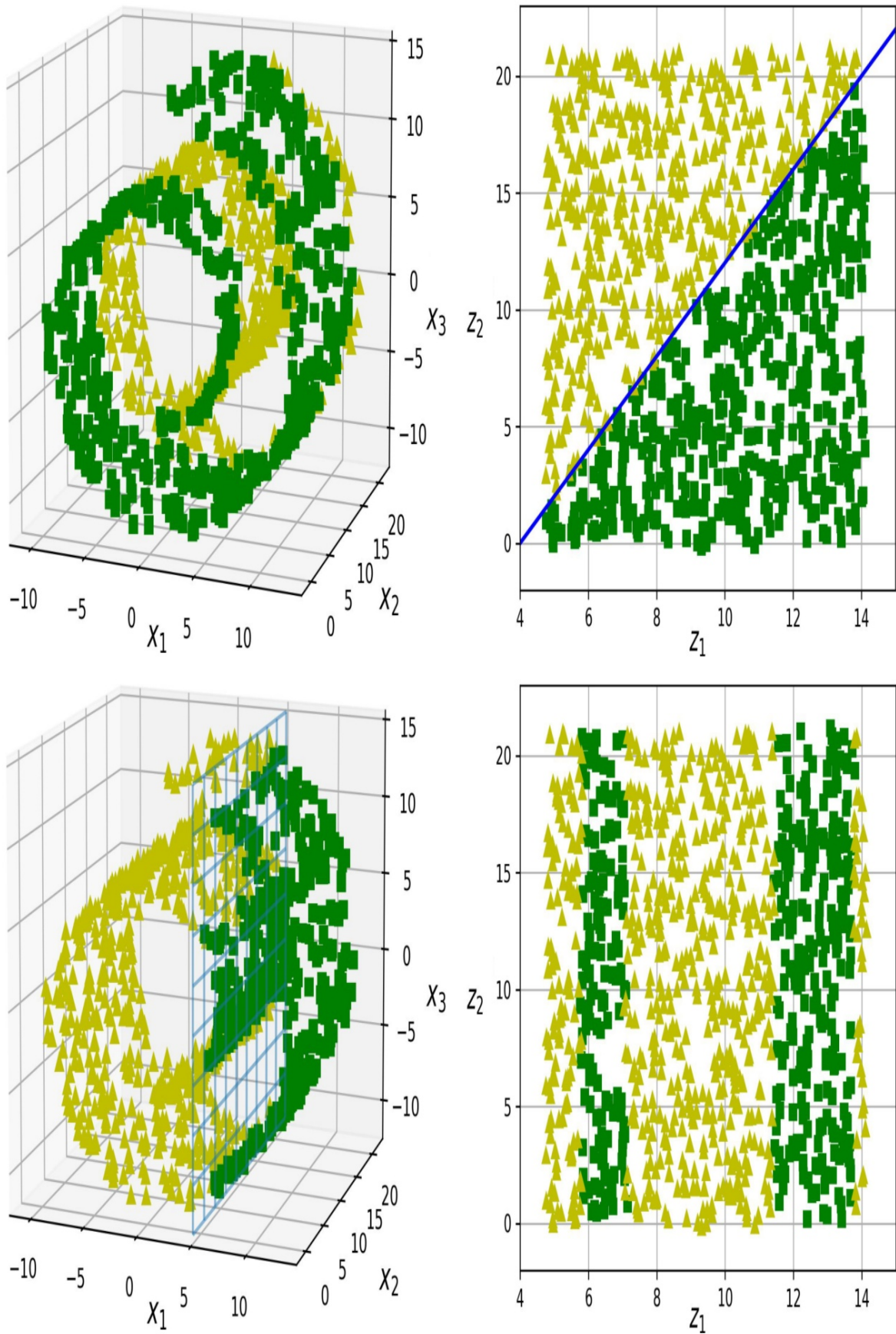


mặt phẳng thẳng đứng), nhưng nó trông phức tạp hơn trong đa tạp trái phẳng (một tập hợp gồm bốn đoạn thẳng độc lập).

Tóm lại, việc giảm chiều dữ liệu huấn luyện trước khi huấn luyện mô hình thường sẽ giúp tăng tốc quá trình huấn luyện, nhưng không phải lúc nào cũng dẫn đến giải pháp tốt hơn hoặc đơn giản hơn; tất cả phụ thuộc vào tập dữ liệu.

Hy vọng giờ đây bạn đã hiểu rõ hơn về vấn đề "lời nguyền của chiều không gian" và cách các thuật toán giảm chiều có thể khắc phục nó, đặc biệt khi giả định về đa tạp được thỏa mãn. Phần còn lại của chương này sẽ trình bày một số thuật toán giảm chiều phổ biến nhất.



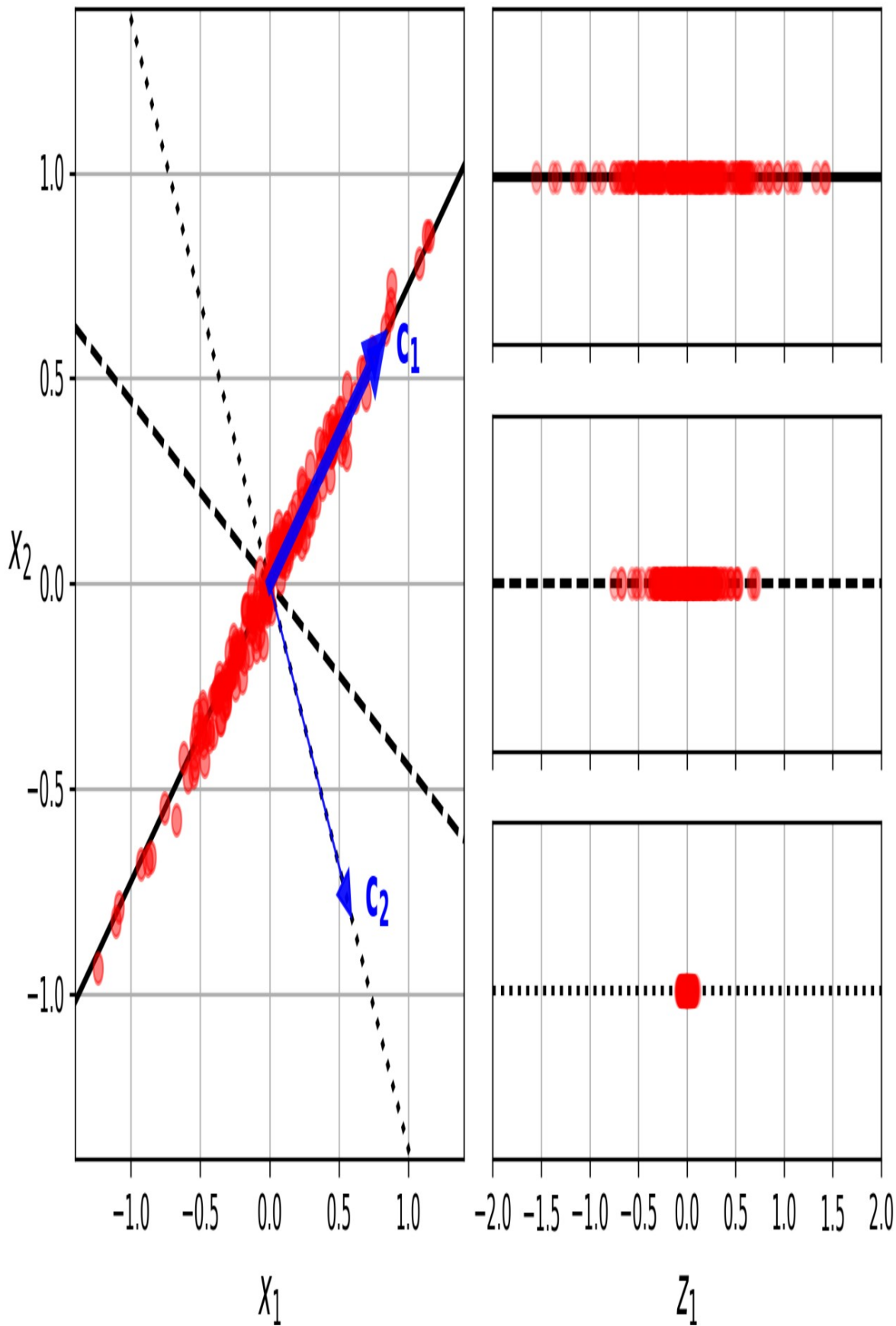
Hình 8-6. Ranh giới quyết định có thể không phải lúc nào cũng đơn giản hơn với số chiều thấp hơn.

PCA

Phân tích thành phần chính (PCA) là thuật toán giảm chiều phổ biến nhất hiện nay. Đầu tiên, nó xác định siêu mặt phẳng nằm gần dữ liệu nhất, sau đó chiếu dữ liệu lên siêu mặt phẳng đó, giống như trong [Hình 8-2](#).

Bảo toàn phương sai

Trước khi có thể chiếu tập dữ liệu huấn luyện lên một siêu mặt phẳng có chiều thấp hơn, trước tiên bạn cần chọn siêu mặt phẳng phù hợp. Ví dụ, một tập dữ liệu 2D đơn giản được biểu diễn ở bên trái trong [Hình 8-7](#), cùng với ba trục khác nhau (tức là các siêu mặt phẳng 1D). Ở bên phải là kết quả của phép chiếu tập dữ liệu lên từng trục này. Như bạn có thể thấy, phép chiếu lên đường liền nét giữ lại độ biến thiên tối đa (trên cùng), trong khi phép chiếu lên đường chấm chấm giữ lại rất ít độ biến thiên (dưới cùng) và phép chiếu lên đường gạch ngang giữ lại một lượng độ biến thiên trung bình (ở giữa).



Hình 8-7. Chọn không gian con để chiếu lên.

Việc chọn trục giữ lại lượng phương sai tối đa có vẻ hợp lý, vì nó có khả năng mất ít thông tin hơn so với các phép chiếu khác. Một cách khác để biện minh cho sự lựa chọn này là đó là trục giúp giảm thiểu khoảng cách bình phương trung bình giữa tập dữ liệu gốc và phép chiếu của nó lên trục đó. Đây chính là ý tưởng khá đơn giản đằng sau PCA.

4

Các thành phần chính

PCA xác định trục chiếm tỷ lệ phương sai lớn nhất trong tập dữ liệu huấn luyện. Trong Hình 8-7, đó là đường liền nét. Nó cũng tìm ra trục thứ hai, vuông góc với trục đầu tiên, chiếm tỷ lệ phương sai còn lại lớn nhất. Trong ví dụ 2D này, không có sự lựa chọn nào khác: đó là đường chấm chấm. Nếu đó là tập dữ liệu có chiều cao hơn, PCA cũng sẽ tìm ra trục thứ ba, vuông góc với cả hai trục trước đó, và trục thứ tư, thứ năm, v.v. – nhiều trục bằng số chiều trong tập dữ liệu.

Trục i^{th} được gọi là thành phần chính (PC) i của dữ liệu. Trong Hình 8-7, PC đầu tiên là trục mà vectơ c nằm trên đó, và PC thứ hai là trục mà vectơ c nằm trên đó. Trong Hình 8-2, hai PC đầu tiên nằm trên mặt phẳng chiếu, và PC thứ ba là trục vuông góc với mặt phẳng đó.

Sau phép chiếu, trong Hình 8-3, PC đầu tiên tương ứng với trục z_1 và PC thứ hai cũng tương ứng với trục z_2 .

GHI CHÚ

Đối với mỗi thành phần chính, PCA tìm một vectơ đơn vị có tâm bằng 0, chỉ theo hướng của thành phần chính đó. Vì hai vectơ đơn vị đối lập nằm trên cùng một trục, hướng của các vectơ đơn vị do PCA trả về không ổn định: nếu bạn thay đổi nhẹ tập dữ liệu huấn luyện và chạy lại PCA, các vectơ đơn vị có thể chỉ theo hướng ngược lại so với các vectơ ban đầu. Tuy nhiên, nhìn chung chúng vẫn nằm trên cùng một trục. Trong một số trường hợp, một cặp vectơ đơn vị thậm chí có thể xoay hoặc hoán đổi vị trí (nếu phương sai dọc theo hai trục này rất gần nhau), nhưng mặt phẳng mà chúng xác định nhìn chung vẫn giữ nguyên.

Vậy làm thế nào để tìm ra các thành phần chính của một tập dữ liệu huấn luyện? May mắn thay, có một kỹ thuật phân tích ma trận chuẩn gọi là Phân tích giá trị đơn (SVD) có thể phân tích ma trận tập dữ liệu huấn luyện X thành phép nhân ma trận của ba ma trận $U \Sigma V$ vectơ, trong đó V chứa đơn vị xác định tất cả các thành phần chính mà chúng ta đang tìm kiếm, như được thể hiện trong **Phương trình 8-1**.

Phương trình 8-1. Ma trận thành phần chính

$$V = \begin{bmatrix} c_1 & c_2 & \dots & c_n \end{bmatrix}$$

Đoạn mã Python sau sử dụng hàm `svd()` của NumPy để thu được tất cả các thành phần chính của tập dữ liệu huấn luyện 3D được biểu diễn trong **Hình 8-2**, sau đó nó trích xuất hai vectơ đơn vị xác định hai thành phần chính đầu tiên:

```
import numpy as np

X = [...] # tạo một tập dữ liệu 3D nhỏ
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt[0]
c2 = Vt[1]
```

CẢNH BÁO

PCA giả định rằng tập dữ liệu được tập trung xung quanh gốc tọa độ. Như chúng ta sẽ thấy, các lớp PCA của Scikit-Learn sẽ tự động thực hiện việc tập trung dữ liệu cho bạn. Nếu bạn tự triển khai PCA (như trong ví dụ trước), hoặc nếu bạn sử dụng các thư viện khác, đừng quên tập trung dữ liệu trước.

Chiếu xuống các chiều d

Sau khi đã xác định tất cả các thành phần chính, bạn có thể giảm chiều dữ liệu xuống còn d chiều bằng cách chiếu nó lên siêu mặt phẳng được xác định bởi d thành phần chính đầu tiên. Việc lựa chọn này

Siêu mặt phẳng đảm bảo phép chiếu sẽ bảo toàn càng nhiều biến thiên càng tốt. Ví dụ, trong **Hình 8-2**, tập dữ liệu 3D được chiếu xuống mặt phẳng 2D được xác định bởi hai thành phần chính đầu tiên, bảo toàn phần lớn biến thiên của tập dữ liệu. Kết quả là, phép chiếu 2D trông rất giống với tập dữ liệu 3D gốc.

Để chiếu tập dữ liệu huấn luyện lên siêu mặt phẳng và thu được tập dữ liệu giảm chiều $X_{d\text{-proj}}$ có kích thước d , hãy tính phép nhân ma trận của ma trận tập dữ liệu huấn luyện X với d cột đầu tiên của V , được định nghĩa là ma trận chứa ma trận W của V , như được thể hiện trong **Phương trình 8-2**.

Phương trình 8-2. Chiếu tập dữ liệu huấn luyện xuống còn d chiều.

$$X_{d\text{-proj}} = XV_d$$

Đoạn mã Python sau đây chiếu tập dữ liệu huấn luyện lên mặt phẳng được xác định bởi hai thành phần chính đầu tiên:

```
W2 = Vt[:2].T
X2D = X_centered @ W2
```

Vậy là xong! Giờ bạn đã biết cách giảm chiều dữ liệu của bất kỳ tập dữ liệu nào bằng cách chiếu nó xuống bất kỳ số chiều nào, đồng thời bảo toàn càng nhiều phương sai càng tốt.

Sử dụng Scikit-Learn

Lớp PCA của Scikit-Learn sử dụng SVD để triển khai PCA, giống như chúng ta đã làm trước đó trong chương này. Đoạn mã sau áp dụng PCA để giảm chiều dữ liệu xuống còn hai chiều (lưu ý rằng nó tự động xử lý việc căn giữa dữ liệu):

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X2D = pca.fit_transform(X)
```

Sau khi áp dụng bộ chuyển đổi PCA cho tập dữ liệu, thuộc tính `components_` của nó chứa ma trận chuyển vị của W : nó chứa một hàng cho mỗi trong số d thành phần chính đầu tiên.

Tỷ lệ phương sai được giải thích

Một thông tin hữu ích khác là tỷ lệ phương sai được giải thích của mỗi thành phần chính, có sẵn thông qua biến

`explained_variance_ratio_`. Tỷ lệ này cho biết tỷ lệ phương sai của tập dữ liệu nằm dọc theo mỗi thành phần chính. Ví dụ, hãy xem xét tỷ lệ phương sai được giải thích của hai thành phần đầu tiên của tập dữ liệu 3D được thể hiện trong **Hình 8-2**:

```
>>> pca.explained_variance_ratio_array
([0.7578477 , 0.15186921])
```

Kết quả này cho thấy khoảng 76% phương sai của tập dữ liệu nằm dọc theo thành phần chính thứ nhất (PC1), và khoảng 15% nằm dọc theo thành phần chính thứ hai (PC2). Điều này để lại khoảng 9% cho thành phần chính thứ ba (PC3), vì vậy có thể giả định rằng PC3 có lẽ mang rất ít thông tin.

Lựa chọn số chiều phù hợp

Thay vì tùy ý chọn số chiều cần giảm xuống, cách đơn giản hơn là chọn số chiều sao cho tổng của chúng chiếm một phần đủ lớn trong phương sai (ví dụ: 95%). Trừ khi, tất nhiên, bạn đang giảm chiều dữ liệu để trực quan hóa dữ liệu, trong trường hợp đó bạn sẽ muốn giảm chiều xuống còn 2 hoặc 3.

Đoạn mã sau tải và chia tập dữ liệu MNIST (được giới thiệu trong...)

Chương 3) và thực hiện PCA mà không giảm chiều, sau đó tính toán số chiều tối thiểu cần thiết để bảo toàn 95% phương sai của tập huấn luyện:

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
```



```

X_train, y_train = mnist.data[:60_000], mnist.target[:60_000]
X_test, y_test = mnist.data[60_000:], mnist.target[60_000:]

pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_) d = np.argmax(cumsum
>= 0.95) + 1 # d bằng 154

```

Bạn có thể đặt `n_components=d` và chạy PCA lại. Nhưng có một lựa chọn tốt hơn: thay vì chỉ định số lượng thành phần chính bạn muốn giữ lại, bạn có thể đặt `n_components` là một số thực nằm giữa 0,0 và 1,0, cho biết tỷ lệ phương sai bạn muốn giữ lại:

```

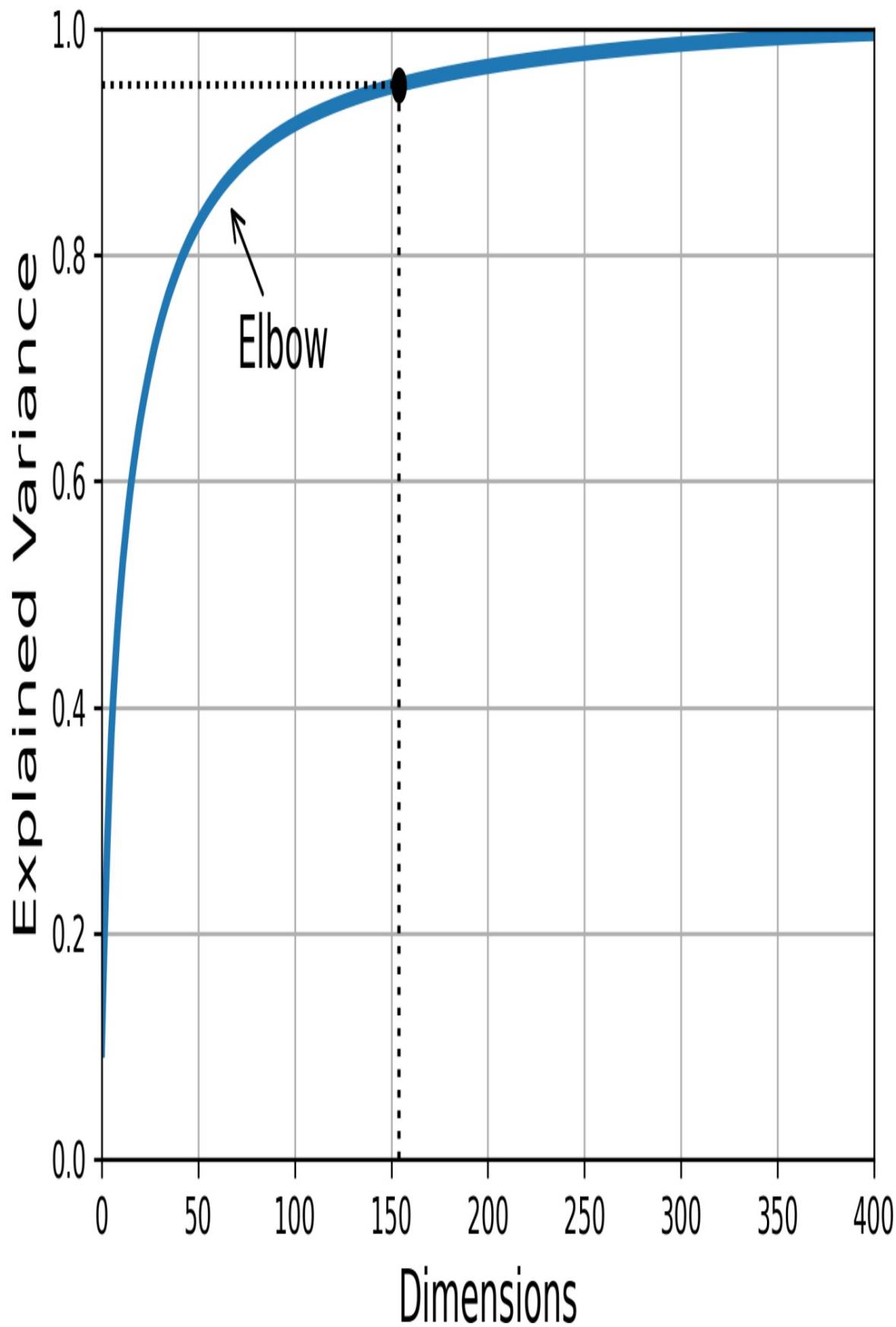
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)

```

Số lượng thành phần thực tế được xác định trong quá trình huấn luyện và được lưu trữ trong thuộc tính `n_components_`:

```
>>> pca.n_components_ 154
```

Một lựa chọn khác nữa là vẽ đồ thị phương sai được giải thích theo số chiều (chỉ cần vẽ đồ thị tổng tích lũy; xem [Hình 8-8](#)). Thường sẽ có một điểm uốn trên đường cong, nơi phương sai được giải thích ngừng tăng nhanh. Trong trường hợp này, bạn có thể thấy rằng việc giảm số chiều xuống khoảng 100 chiều sẽ không làm mất quá nhiều phương sai được giải thích.



Hình 8-8. Phương sai được giải thích như một hàm của số chiều

Cuối cùng, nếu bạn sử dụng giảm chiều dữ liệu như một bước tiền xử lý cho một tác vụ học có giám sát (ví dụ: phân loại), thì bạn có thể điều chỉnh số chiều giống như bất kỳ siêu tham số nào khác (xem [Chương 2](#)). Ví dụ, đoạn mã sau tạo ra một quy trình hai bước, đầu tiên giảm chiều dữ liệu bằng PCA, sau đó phân loại bằng Rừng ngẫu nhiên. Tiếp theo, nó sử dụng RandomizedSearchCV để tìm ra sự kết hợp tốt các siêu tham số cho cả PCA và bộ phân loại Rừng ngẫu nhiên. Ví dụ này thực hiện tìm kiếm nhanh, chỉ điều chỉnh hai siêu tham số, huấn luyện trên chỉ 1.000 trường hợp và chạy chỉ 10 lần lặp, nhưng bạn có thể thực hiện tìm kiếm kỹ lưỡng hơn nếu có thời gian.

```
from sklearn.ensemble import RandomForestClassifier from
sklearn.model_selection import RandomizedSearchCV from sklearn.pipeline
import make_pipeline

clf = make_pipeline(PCA(random_state=42),
                    RandomForestClassifier(random_state=42))
param_distrib =
    { "pca__n_components": np.arange(10, 80),
      "randomforestclassifier__n_estimators": np.arange(50, 500)

} rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,

                                random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])
```

Hãy cùng xem xét các siêu tham số tốt nhất đã tìm thấy:

```
>>> print(rnd_search.best_params_)
{'randomforestclassifier__n_estimators': 465,
 'pca__n_components': 23}
```

Điều thú vị là số lượng thành phần tối ưu lại thấp đến vậy: chúng ta đã giảm một tập dữ liệu 784 chiều xuống chỉ còn 23 chiều! Điều này liên quan đến việc chúng ta đã sử dụng mô hình Rừng ngẫu nhiên (Random Forest), một mô hình khá mạnh mẽ. Nếu chúng ta sử dụng một mô hình tuyến tính khác, chẳng hạn như SGDClassifier, thì quá trình tìm kiếm sẽ cho thấy chúng ta cần giữ lại nhiều chiều hơn (khoảng 70).

PCA để nén

Sau khi giảm chiều dữ liệu, tập dữ liệu huấn luyện chiếm ít không gian hơn nhiều. Ví dụ, sau khi áp dụng PCA cho tập dữ liệu MNIST trong khi vẫn giữ lại 95% phương sai, chúng ta chỉ còn lại 154 đặc trưng, thay vì 784 đặc trưng ban đầu. Như vậy, tập dữ liệu giờ đây chỉ còn chứa đến 20% kích thước ban đầu, và chúng ta chỉ mất đi 5% phương sai! Đây là tỷ lệ nén hợp lý, và dễ dàng nhận thấy việc giảm kích thước như vậy sẽ giúp tăng tốc thuật toán phân loại một cách đáng kể.

Cũng có thể giải nén tập dữ liệu đã giảm kích thước trở lại 784 chiều bằng cách áp dụng phép biến đổi nghịch đảo của phép chiếu PCA.

Điều này sẽ không trả lại cho bạn dữ liệu gốc, vì phép chiếu đã làm mất một phần thông tin (trong phạm vi 5% phương sai bị loại bỏ), nhưng kết quả có thể sẽ gần với dữ liệu gốc. Khoảng cách bình phương trung bình giữa dữ liệu gốc và dữ liệu được tái tạo (đã được nén và giải nén) được gọi là sai số tái tạo.

Phương thức `inverse_transform()` cho phép chúng ta giải nén dữ liệu đã được giảm bớt. Tập dữ liệu MNIST được khôi phục về 784 chiều:

```
X_recovered = pca.inverse_transform(X_reduced)
```

Hình 8-9 hiển thị một vài chữ số từ tập dữ liệu huấn luyện ban đầu (bên trái) và các chữ số tương ứng sau khi nén và giải nén. Bạn có thể thấy rằng chất lượng hình ảnh bị giảm nhẹ, nhưng các chữ số vẫn còn nguyên vẹn phần lớn.

Original

Compressed

4 2 9 3 /

4 2 9 3 /

5 7 1 4 3

5 7 1 4 3

7 9 7 0 8

7 9 7 0 8

0 9 9 1 4

0 9 9 1 4

5 1 7 6 1

5 1 7 6 1

Hình 8-9. Nén MNIST giữ lại 95% phương sai

Phương trình của phép biến đổi nghịch đảo được thể hiện trong **Phương trình 8-3**.

Phương trình 8-3. Biến đổi ngược PCA, trở lại số chiều ban đầu.

$$X_{\text{recovered}} = X_d \text{-proj} W_d$$

PCA ngẫu nhiên

Nếu bạn đặt siêu tham số `svd_solver` thành "randomized", Scikit-Learn sẽ sử dụng thuật toán ngẫu nhiên gọi là PCA ngẫu nhiên (Randomized PCA) để nhanh chóng tìm ra xấp xỉ của d thành phần chính đầu tiên. Độ phức tạp tính toán của nó là $O(m \times d) + O(d^2)$, thay vì $O(m \times n) + O(n^3)$ đối với phương pháp SVD đầy đủ, do đó nó nhanh hơn đáng kể so với SVD đầy đủ khi d nhỏ hơn nhiều so với n :

```
rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)
```

```
X_reduced = rnd_pca.fit_transform(X_train)
```

MẸO

Theo mặc định, `svd_solver` thực tế được đặt thành "auto": Scikit-Learn tự động sử dụng thuật toán PCA ngẫu nhiên nếu $\max(m, n) > 500$ và `n_components` là một số nguyên nhỏ hơn 80% của $\min(m, n)$, nếu không thì nó sẽ sử dụng phương pháp SVD đầy đủ. Vì vậy, đoạn mã trước đó sẽ sử dụng thuật toán PCA ngẫu nhiên ngay cả khi bạn đã xóa đối số `svd_solver="randomized"`, vì $154 < 0,8 \times 784$. Nếu bạn muốn buộc Scikit-Learn sử dụng SVD đầy đủ để có kết quả chính xác hơn một chút, bạn có thể đặt siêu tham số `svd_solver` thành "full".

PCA gia tăng

Một vấn đề với các phương pháp PCA trước đây là chúng yêu cầu toàn bộ tập dữ liệu huấn luyện phải nằm gọn trong bộ nhớ để thuật toán có thể hoạt động. May mắn thay, các thuật toán PCA gia tăng (IPCA) đã được phát triển.

Chúng cho phép bạn chia tập dữ liệu huấn luyện thành các lô nhỏ và đưa vào IPCA.

Thuật toán xử lý từng mini-batch một. Điều này hữu ích cho các tập dữ liệu huấn luyện lớn và để áp dụng PCA trực tuyến (tức là, tức thời, khi các trường hợp mới xuất hiện).

Đoạn mã sau chia tập dữ liệu huấn luyện MNIST thành 100 mini-batch (sử dụng hàm `array_split()` của NumPy) và đưa chúng vào Scikit-

Lớp học `IncrementalPCA` của Learn để giảm số chiều của tập dữ liệu MNIST xuống còn 154 chiều, giống như trước đây. Lưu ý rằng bạn phải gọi phương thức `partial_fit()` với mỗi mini-batch, thay vì phương thức `fit()` với toàn bộ tập huấn luyện:

```
from sklearn.decomposition import IncrementalPCA

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)
for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

X_reduced = inc_pca.transform(X_train)
```

Ngoài ra, bạn cũng có thể sử dụng lớp `memmap` của NumPy, cho phép bạn thao tác với một mảng lớn được lưu trữ trong tệp nhị phân trên đĩa như thể toàn bộ mảng đó nằm trong bộ nhớ; lớp này chỉ tải dữ liệu cần thiết vào bộ nhớ khi cần. Để minh họa điều này, trước tiên hãy tạo một tệp `memmap` và sao chép tập dữ liệu huấn luyện MNIST vào đó, sau đó gọi hàm `flush()` để đảm bảo rằng bất kỳ dữ liệu nào vẫn còn trong bộ nhớ cache đều được lưu vào đĩa. Trong thực tế, `X_train` thường không vừa trong bộ nhớ nên bạn sẽ tải nó từng phần và lưu mỗi phần vào đúng vị trí của mảng `memmap`:

```
tên_tệp = "my_mnist.mmap"
X_mmap = np.memmap(filename, dtype='float32', mode='write', shape=X_train.shape)

X_mmap[:] = X_train # có thể là một vòng lặp thay thế, lưu trữ từng khối dữ liệu một
X_mmap.flush()
```

Tiếp theo, chúng ta có thể tải tệp `memmap` và sử dụng nó như một mảng NumPy thông thường. Hãy sử dụng lớp `IncrementalPCA` để giảm chiều dữ liệu. Vì thuật toán này chỉ sử dụng một phần nhỏ của mảng tại bất kỳ thời điểm nào, nên bộ nhớ sẽ được tối ưu hóa.

Việc sử dụng vẫn được kiểm soát. Điều này cho phép gọi phương thức `fit()` thông thường thay vì `partial_fit()`, khá tiện lợi:

```
X_mmap = np.memmap(filename, dtype="float32",
mode="readonly").reshape(-1, 784)
batch_size = X_mmap.shape[0] // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)
inc_pca.fit(X_mmap)
```

CẢNH BÁO

Chỉ dữ liệu nhị phân thô được lưu vào đĩa, vì vậy bạn cần chỉ định kiểu dữ liệu và kích thước của mảng khi tải nó. Nếu bạn bỏ qua kích thước, `np.memmap()` sẽ trả về một mảng 1 chiều.

Đối với các tập dữ liệu có chiều rất cao, PCA có thể quá chậm. Như chúng ta đã thấy trước đó, ngay cả khi bạn sử dụng PCA ngẫu nhiên, độ phức tạp tính toán của nó vẫn là $O(m^2 \times d) + O(d)$, vì vậy số chiều mục tiêu d không được quá lớn. Nếu bạn đang xử lý một tập dữ liệu có hàng chục nghìn đặc trưng trở lên (ví dụ: hình ảnh), thì quá trình huấn luyện có thể trở nên quá chậm: trong trường hợp này, bạn nên xem xét sử dụng Phép chiếu ngẫu nhiên thay thế.

Phép chiếu ngẫu nhiên

Như tên gọi của nó, thuật toán Chiếu ngẫu nhiên (Random Projection) chiếu dữ liệu xuống không gian có chiều thấp hơn bằng cách sử dụng phép chiếu tuyến tính ngẫu nhiên. Điều này nghe có vẻ điên rồ, nhưng hóa ra phép chiếu ngẫu nhiên như vậy lại rất có khả năng bảo toàn khoảng cách khá tốt, như đã được William B. Johnson và Joram Lindenstrauss chứng minh bằng toán học trong một bài báo nổi tiếng. Vì vậy, hai trường hợp tương tự sẽ vẫn tương tự sau khi chiếu, và hai trường hợp rất khác nhau sẽ vẫn rất khác nhau.

Rõ ràng, càng giảm số chiều, càng nhiều thông tin bị mất và khoảng cách càng bị sai lệch. Vậy làm thế nào để chọn số chiều tối ưu? Johnson và Lindenstrauss đã đưa ra một phương trình xác định số chiều tối thiểu cần bảo toàn trong không gian ba chiều.

Để đảm bảo—với xác suất cao—khoảng cách sẽ không thay đổi quá một mức dung sai nhất định. Ví dụ, nếu bạn có một tập dữ liệu chứa $m = 5.000$ trường hợp với $n = 20.000$ đặc trưng mỗi trường hợp, và bạn không muốn khoảng cách bình phương giữa bất kỳ hai trường hợp nào thay đổi quá $\epsilon = 10\%$, thì bạn nên chiếu dữ liệu xuống d chiều, với $d \geq 4 \log(m) / (\frac{1}{2} \epsilon^2 - \frac{1}{3} \epsilon^3)$, tức là 7.300 chiều. Đó là một sự giảm chiều đáng kể! Lưu ý rằng phương trình không sử dụng n , nó chỉ dựa vào m và ϵ . Phương trình này được thực hiện bởi hàm `johnson_lindenstrauss_min_dim()`:

```
>>> from sklearn.random_projection import
johnson_lindenstrauss_min_dim >>> m, ε
= 5_000, 0.1 >>> d =
johnson_lindenstrauss_min_dim(m, eps=ε) >>> d

7300
```

Giờ đây, chúng ta có thể tạo ra một ma trận ngẫu nhiên P có kích thước $[d, n]$, trong đó mỗi phần tử được lấy mẫu ngẫu nhiên từ phân phối Gauss với giá trị trung bình là 0 và phương sai là $1/d$, và chúng ta sử dụng nó để chiếu một tập dữ liệu từ n chiều xuống d chiều:

```
n = 20_000
np.random.seed(42)
P = np.random.randn(d, n) / np.sqrt(d) # độ lệch chuẩn = căn bậc hai của phương sai

X = np.random.randn(m, n) # tạo một tập dữ liệu giả X_reduced = X @ PT
```

Chỉ đơn giản vậy thôi! Nó rất dễ sử dụng và hiệu quả, không cần huấn luyện: thuật toán chỉ cần hình dạng của tập dữ liệu để tạo ra ma trận ngẫu nhiên, thế thôi. Bản thân dữ liệu không được sử dụng chút nào.

Scikit-Learn cung cấp lớp `GaussianRandomProjection` để thực hiện chính xác những gì chúng ta vừa làm: khi bạn gọi phương thức `fit()` của nó, nó sử dụng `johnson_lindenstrauss_min_dim()` để xác định chiều dữ liệu đầu ra, sau đó nó tạo ra một ma trận ngẫu nhiên, mà nó lưu trữ trong

thuộc tính `components_`. Sau đó, khi bạn gọi `transform()`, nó sẽ sử dụng ma trận này để thực hiện phép chiếu. Khi tạo bộ chuyển đổi, bạn có thể đặt `eps` nếu muốn điều chỉnh ϵ (mặc định là 0.1) và `n_components` nếu muốn buộc một chiều mục tiêu cụ thể `d`. Ví dụ mã sau cho kết quả tương tự như trên (bạn cũng có thể xác minh rằng `gaussian_rnd_proj.components_` bằng `P`):

```
from sklearn.random_projection import GaussianRandomProjection

gaussian_rnd_proj = GaussianRandomProjection(eps=ε,
random_state=42)
X_reduced = gaussian_rnd_proj.fit_transform(X) # kết quả tương tự như trên
```

Scikit-Learn cung cấp một bộ chuyển đổi chiều ngẫu nhiên thứ hai: `SparseRandomProjection`. Nó xác định chiều mục tiêu theo cùng một cách, tạo ra một ma trận ngẫu nhiên có cùng hình dạng và thực hiện phép chiếu giống hệt nhau. Sự khác biệt chính là ma trận ngẫu nhiên này thưa. Điều này có nghĩa là nó sử dụng ít bộ nhớ hơn nhiều: khoảng 25 MB thay vì gần 1,2 GB trong ví dụ trước! Và nó cũng nhanh hơn nhiều, cả trong việc tạo ma trận ngẫu nhiên và giảm chiều: nhanh hơn khoảng 50% trong ví dụ trước. Hơn nữa, nếu đầu vào thưa, phép biến đổi sẽ giữ nguyên tính chất thưa của nó (trừ khi bạn đặt `dense_output=True`). Cuối cùng, nó có cùng thuộc tính bảo toàn khoảng cách như phương pháp trước đó, và chất lượng giảm chiều là tương đương. Tóm lại, thường nên sử dụng bộ chuyển đổi này thay vì bộ chuyển đổi đầu tiên, đặc biệt là đối với các tập dữ liệu lớn hoặc thưa.

Tỷ lệ r của các phần tử khác 0 trong ma trận ngẫu nhiên thưa được gọi là mật độ của nó. Theo mặc định, nó bằng $1/\sqrt{n}$. Với 20.000 đặc trưng, điều này có nghĩa là chỉ có một trong ~141 ô trong ma trận ngẫu nhiên là khác 0: điều đó khá thưa thớt! Bạn có thể đặt tham số siêu tham số mật độ thành một giá trị khác nếu muốn. Mỗi ô trong ma trận ngẫu nhiên thưa có xác suất r là khác 0, và mỗi giá trị khác 0 là $-v$ hoặc $+v$ (cả hai đều có xác suất như nhau), trong đó $v = 1/\sqrt{dr}$.

GHI CHÚ

Phép chiếu ngẫu nhiên không phải lúc nào cũng được sử dụng để giảm chiều dữ liệu của các tập dữ liệu lớn. Ví dụ, một bài báo năm 2017⁷ Nghiên cứu của Sanjoy Dasgupta và cộng sự đã chỉ ra rằng não bộ của ruồi giấm thực hiện một cơ chế tương tự như Phép chiếu ngẫu nhiên để ánh xạ các tín hiệu khứu giác chiều thấp dày đặc thành các tín hiệu nhị phân chiều cao thưa thớt: đối với mỗi mùi hương, chỉ một phần nhỏ các tế bào thần kinh đầu ra được kích hoạt, nhưng các mùi hương tương tự lại kích hoạt nhiều tế bào thần kinh giống nhau. Điều này tương tự như một thuật toán nổi tiếng gọi là Bấm nhảy cảm vị trí (LSH), thường được sử dụng trong các công cụ tìm kiếm để nhóm các tài liệu tương tự nhau.

Nếu bạn muốn thực hiện phép biến đổi ngược, trước tiên bạn cần tính toán ma trận giả nghịch đảo của ma trận thành phần bằng cách sử dụng hàm `pinv()` của SciPy, sau đó nhân dữ liệu đã được rút gọn với ma trận chuyển vị của ma trận giả nghịch đảo:

```
components_pinv = np.linalg.pinv(gaussian_rnd_proj.components_)
X_recovered = X_reduced @ components_pinv.T
```

CẢNH BÁO

Việc tính toán nghịch đảo giả có thể mất rất nhiều thời gian nếu ma trận thành phần lớn, vì độ phức tạp tính toán của `pinv()` là $O(dn^2)$ nếu $d < n$, hoặc $O(nd^2)$ nếu ngược lại.

Tóm lại, Phép chiếu ngẫu nhiên là một thuật toán giảm chiều đơn giản, nhanh, tiết kiệm bộ nhớ và mạnh mẽ đáng ngạc nhiên mà bạn nên ghi nhớ, đặc biệt khi làm việc với các tập dữ liệu có chiều cao.

LLE

Nhúng tuyến tính cục bộ (LLE)⁸ LLE là một kỹ thuật giảm chiều phi tuyến tính (NLDR). Nó là một kỹ thuật Học đa tạp không dựa vào phép chiếu, không giống như PCA và Phép chiếu ngẫu nhiên. Tóm lại, LLE hoạt động bằng cách đầu tiên đo lường mối quan hệ tuyến tính của mỗi trường hợp huấn luyện với các láng giềng gần nhất của nó, và sau đó tìm kiếm một biểu diễn chiều thấp của tập huấn luyện trong đó các mối quan hệ cục bộ này được bảo tồn tốt nhất (xem thêm

(Chi tiết sẽ được công bố sớm). Phương pháp này đặc biệt hiệu quả trong việc trải phẳng các đường cong xoắn, nhất là khi không có quá nhiều nhiễu.

Đoạn mã sau tạo ra một cuộn bánh mì Thụy Sĩ, sau đó sử dụng thư viện Scikit-Learn.

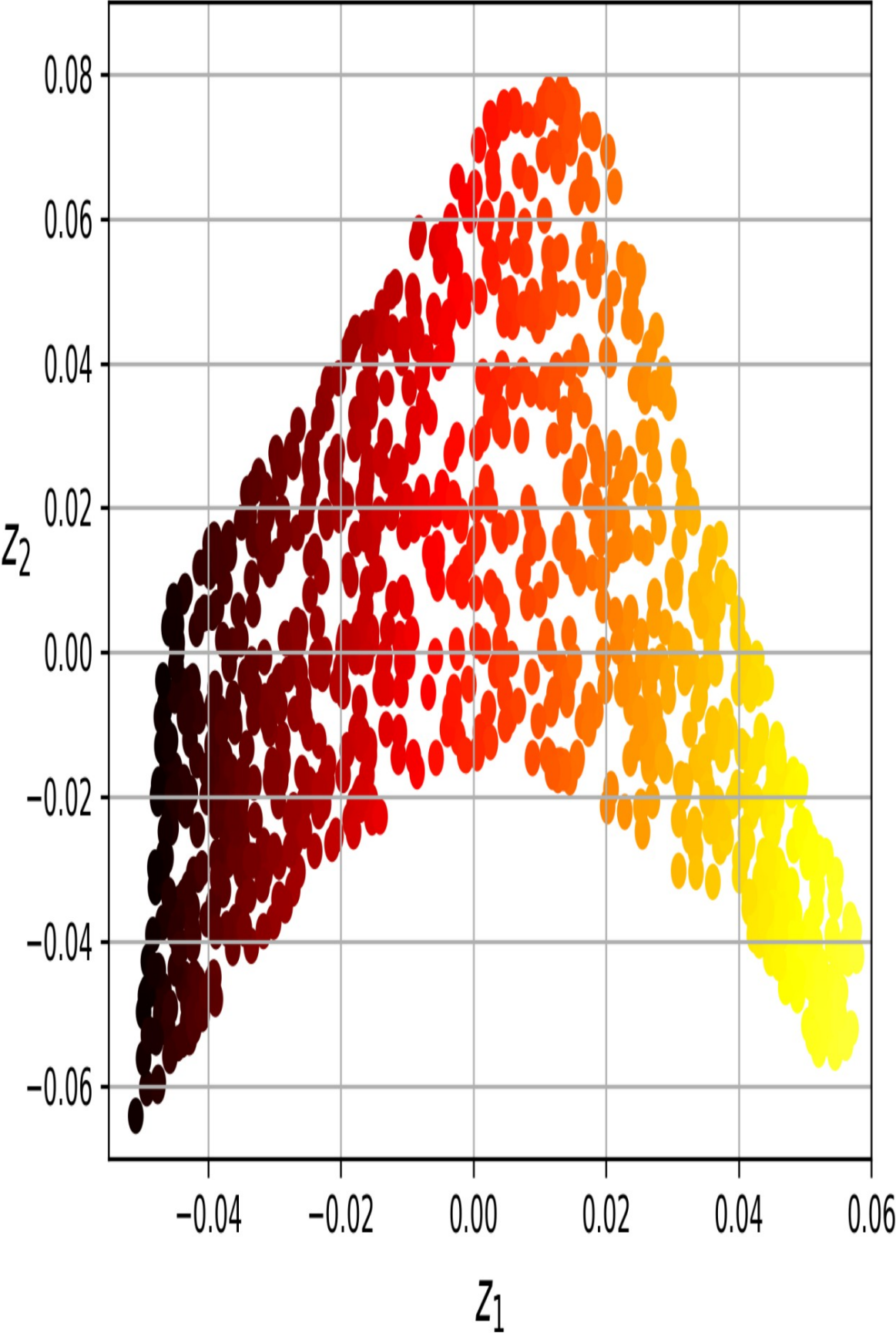
Lớp LocallyLinearEmbedding để mở rộng nó:

```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding

X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2,
                             random_state=42)
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10,
                             random_state=42)
X_unrolled = lle.fit_transform(X_swiss)
```

Biến `t` là một mảng NumPy 1 chiều chứa vị trí của mỗi phần tử dọc theo trục cuộn của bánh mì Thụy Sĩ. Chúng ta không sử dụng nó trong ví dụ này, nhưng nó có thể được sử dụng làm mục tiêu cho một bài toán hồi quy phi tuyến tính.

Tập dữ liệu 2D thu được được hiển thị trong **Hình 8-10**. Như bạn có thể thấy, cuộn bánh mì Thụy Sĩ được trải phẳng hoàn toàn, và khoảng cách giữa các phần tử được bảo toàn tốt ở phạm vi cục bộ. Tuy nhiên, khoảng cách không được bảo toàn ở quy mô lớn hơn: cuộn bánh mì Thụy Sĩ khi trải phẳng phải là hình chữ nhật, chứ không phải là dạng dải bị kéo giãn và xoắn như thế này. Tuy nhiên, LLE đã làm khá tốt trong việc mô hình hóa đa tạp này.



Hình 8-10. Cuộn bánh mì Thụy Sĩ được trải phẳng bằng phương pháp LLE.

Đây là cách LLE hoạt động: với mỗi trường hợp huấn luyện x , nó xác định k láng giềng gần nhất của nó (trong đoạn mã trước đó $k = 10$), sau đó cố gắng tái tạo x như một hàm tuyến tính của các láng giềng này. Cụ thể hơn, nó cố gắng tìm các trọng số w sao cho bình phương khoảng cách giữa một trong m và w_i là nhỏ nhất có thể, giả sử $w = 0$ nếu x không phải là (i) k láng giềng gần nhất của x . Như vậy, bước đầu tiên của LLE là sự ràng buộc. trong bài toán tối ưu hóa được mô tả trong **Phương trình 8-4**, trong đó W là ma trận trọng số chứa tất cả các trọng số $w_{i,j}$. Ràng buộc thứ hai đơn giản là (i) chuẩn hóa trọng số cho mỗi trường hợp huấn luyện x .

Phương trình 8-4. Bước một của LLE: mô hình hóa tuyến tính các mối quan hệ cục bộ

$$W^* = \underset{W}{\operatorname{argmin}} \sum_{i=1}^m \left(x^{(i)} - \sum_{j=1}^n w_{i,j} x^{(j)} \right)^2$$

nếu $x^{(j)}$ không phải là một trong k nn của $x^{(i)}$

subject to $\{ w_{i,j} = 0, j = 1 \text{ cho } i = 1, 2, \dots, m$

Sau bước này, ma trận trọng số W^* (chứa các trọng số $w^*_{i,j}$) mã hóa các mối quan hệ tuyến tính cục bộ giữa các trường hợp huấn luyện. Bước thứ hai là ánh xạ các trường hợp huấn luyện vào không gian d chiều (trong đó $d < n$) đồng thời bảo toàn các mối quan hệ cục bộ này càng nhiều càng tốt. Nếu z là ảnh của x trong không gian d chiều này, thì chúng ta muốn bình phương của cách giữa z và m này dẫn đến $w^*_{i,j} z^{(j)}$ càng nhỏ càng tốt. Ý tưởng về khoảng bài toán tối ưu hóa không ràng buộc được mô tả trong **Phương trình 8-5**. Nó trông rất giống bước đầu tiên, nhưng thay vì giữ nguyên các trường hợp và tìm trọng số tối ưu, chúng ta đang làm ngược lại: giữ nguyên trọng số và tìm vị trí tối ưu của hình ảnh các trường hợp trong (i) không gian chiều thấp. Lưu ý rằng Z là ma trận chứa tất cả z .

Phương trình 8-5. Bước thứ hai của LLE: giảm chiều trong khi vẫn bảo toàn các mối quan hệ

$$\hat{Z} = \underset{Z}{\operatorname{argmin}} \sum_{i=1}^m \sum_{j=1}^m w_{i,j}^2 \|z^{(i)} - z^{(j)}\|^2$$

Việc triển khai LLE của Scikit-Learn có độ phức tạp tính toán như sau: $O(m \log(m)n \log(k))$ để tìm k láng giềng gần nhất, $O(mnk)$ để tối ưu hóa trọng số và $O(dm)$ để xây dựng các biểu diễn chiều thấp. Thật không may, m trong số hạng cuối cùng khiến thuật toán này có khả năng mở rộng kém đối với các tập dữ liệu rất lớn.

Như bạn thấy, LLE khá khác biệt so với các kỹ thuật chiếu, và nó phức tạp hơn đáng kể, nhưng nó cũng có thể xây dựng các biểu diễn chiều thấp tốt hơn nhiều, đặc biệt nếu dữ liệu không tuyến tính.

Các kỹ thuật giảm chiều khác

Trước khi kết thúc chương này, chúng ta hãy cùng điểm qua một vài kỹ thuật giảm chiều dữ liệu phổ biến khác có sẵn trong Scikit-Learn:

`sklearn.manifold.MDS`

Phương pháp đa chiều (Multidimensional Scaling - MDS) giảm số chiều dữ liệu trong khi vẫn cố gắng bảo toàn khoảng cách giữa các phần tử. Phép chiếu ngẫu nhiên (Random Projection) thực hiện điều này đối với dữ liệu đa chiều, nhưng lại không hiệu quả với dữ liệu ít chiều.

`sklearn.manifold.Isomap`

Isomap tạo ra một đồ thị bằng cách kết nối mỗi phần tử với các phần tử lân cận gần nhất, sau đó giảm chiều dữ liệu trong khi cố gắng bảo toàn khoảng cách trắc địa giữa các phần tử. Khoảng cách trắc địa giữa hai nút trong đồ thị là số nút trên đường đi ngắn nhất giữa hai nút này.

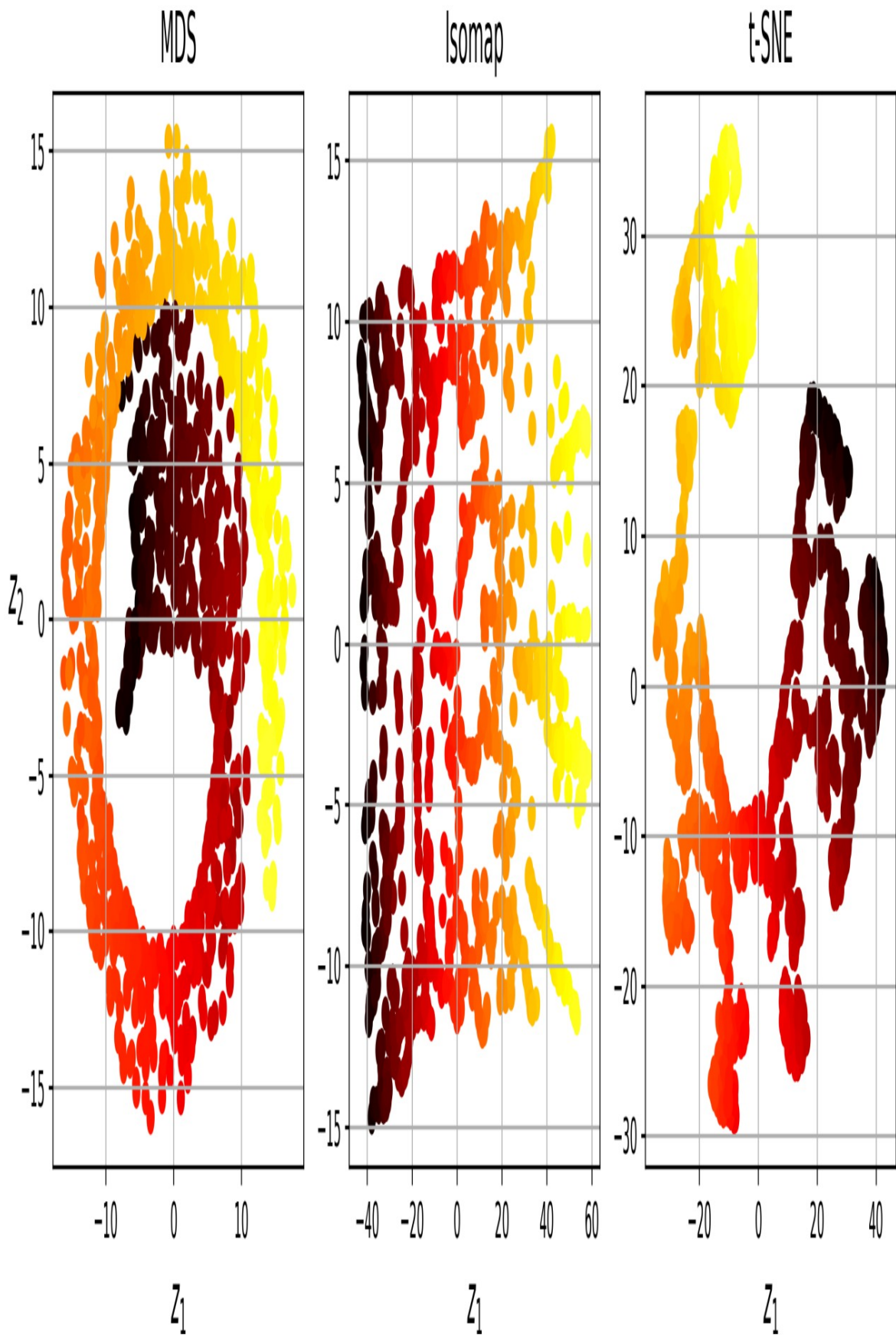
`sklearn.manifold.TSNE`

Phương pháp nhúng lân cận ngẫu nhiên phân tán t (t-Distributed Stochastic Neighbor Embedding - t-SNE) giảm chiều dữ liệu trong khi vẫn cố gắng giữ các trường hợp tương tự gần nhau và các trường hợp khác nhau cách xa nhau. Phương pháp này chủ yếu được sử dụng để trực quan hóa, đặc biệt là để trực quan hóa các cụm trường hợp trong không gian đa chiều. Ví dụ, trong các bài tập ở cuối chương này, bạn sẽ sử dụng t-SNE để trực quan hóa bản đồ 2D của các hình ảnh MNIST.

`sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

Phân tích phân biệt tuyến tính (LDA) là một thuật toán phân loại tuyến tính, và trong quá trình huấn luyện, nó học được các trục phân biệt nhất giữa các lớp. Sau đó, các trục này có thể được sử dụng để xác định một siêu mặt phẳng để chiếu dữ liệu lên đó. Lợi ích của phương pháp này là phép chiếu sẽ giữ cho các lớp cách xa nhau nhất có thể, vì vậy LDA là một kỹ thuật tốt để giảm chiều dữ liệu trước khi chạy một thuật toán phân loại khác (trừ khi chỉ riêng LDA là đủ).

Hình 8-11 thể hiện kết quả của MDS, Isomap và t-SNE trên cuộn bánh mì Thụy Sĩ. MDS có thể làm phẳng cuộn Thụy Sĩ mà không làm mất đi độ cong tổng thể, trong khi Isomap lại loại bỏ hoàn toàn độ cong này. Tùy thuộc vào tác vụ tiếp theo, việc bảo tồn cấu trúc quy mô lớn có thể tốt hoặc xấu. Đối với t-SNE, nó đã làm khá tốt việc làm phẳng cuộn Thụy Sĩ, bảo tồn một chút độ cong, và nó cũng khuếch đại các cụm, làm rách cuộn. Một lần nữa, điều này có thể tốt hoặc xấu, tùy thuộc vào tác vụ tiếp theo.



Hình 8-11. Sử dụng các kỹ thuật khác nhau để thu nhỏ cuộn bánh mì Thụy Sĩ thành hình 2D.

Bài tập

1. Những động lực chính nào thúc đẩy việc giảm chiều dữ liệu?
Những nhược điểm chính là gì?
2. Lời nguyên của chiều không gian là gì?
3. Sau khi giảm chiều dữ liệu, liệu có thể...
Đảo ngược thao tác? Nếu có, bằng cách nào? Nếu không, tại sao?
4. Liệu PCA có thể được sử dụng để giảm chiều dữ liệu của một mô hình phi tuyến tính cao hay không?
bộ dữ liệu?
5. Giả sử bạn thực hiện phân tích thành phần chính (PCA) trên một tập dữ liệu có 1.000 chiều, với tỷ lệ phương sai được giải thích là 95%. Tập dữ liệu kết quả sẽ có bao nhiêu chiều?
6. Trong những trường hợp nào bạn sẽ sử dụng PCA thông thường, PCA tăng dần?
PCA ngẫu nhiên, hay phép chiếu ngẫu nhiên?
7. Làm thế nào bạn có thể đánh giá hiệu suất của việc giảm chiều dữ liệu?
Thuật toán trên tập dữ liệu của bạn là gì?
8. Liệu việc kết hợp hai phương pháp giảm chiều khác nhau có hợp lý không?
thuật toán?
9. Tải bộ dữ liệu MNIST (đã giới thiệu trong [Chương 3](#)) và chia nó thành tập huấn luyện và tập kiểm tra (lấy 60.000 mẫu đầu tiên để huấn luyện và 10.000 mẫu còn lại để kiểm tra). Huấn luyện một bộ phân loại Rừng ngẫu nhiên trên bộ dữ liệu và đo thời gian huấn luyện, sau đó đánh giá mô hình thu được trên tập kiểm tra. Tiếp theo, sử dụng PCA để giảm chiều dữ liệu, với tỷ lệ phương sai được giải thích là 95%. Huấn luyện một bộ phân loại Rừng ngẫu nhiên mới trên bộ dữ liệu đã giảm chiều và xem thời gian huấn luyện. Quá trình huấn luyện có nhanh hơn nhiều không? Tiếp theo, đánh giá bộ phân loại trên tập kiểm tra.

Bộ dữ liệu thử nghiệm. So với bộ phân loại trước thì sao? Thử lại với `SGDClassifier`. PCA giúp ích được bao nhiêu trong trường hợp này?

10. Sử dụng t-SNE để giảm số lượng 5.000 hình ảnh đầu tiên của tập dữ liệu MNIST.

Giảm chiều dữ liệu xuống còn hai chiều và vẽ kết quả bằng Matplotlib. Bạn có thể sử dụng biểu đồ phân tán với 10 màu khác nhau để biểu thị lớp mục tiêu của mỗi hình ảnh. Hoặc, bạn có thể thay thế mỗi điểm trong biểu đồ phân tán bằng lớp tương ứng của đối tượng (một chữ số từ 0 đến 9), hoặc thậm chí vẽ các phiên bản thu nhỏ của chính các hình ảnh chữ số (nếu bạn vẽ tất cả các chữ số, hình ảnh trực quan sẽ quá rối rắm, vì vậy bạn nên lấy mẫu ngẫu nhiên hoặc chỉ vẽ một đối tượng nếu không có đối tượng nào khác đã được vẽ ở khoảng cách gần đó). Bạn sẽ nhận được một hình ảnh trực quan đẹp với các cụm chữ số được phân tách rõ ràng. Hãy thử sử dụng các thuật toán giảm chiều khác như PCA, LLE hoặc MDS và so sánh các hình ảnh trực quan thu được.

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://hom1.info/colab3>.

-
1. Vâng, bốn chiều nếu tính cả thời gian, và thêm vài chiều nữa nếu bạn là nhà lý thuyết dây.
 2. Xem hình chiếu khối lập phương bốn chiều xoay tròn trong không gian 3D tại <https://hom1.info/30>. Hình ảnh do người dùng Wikipedia NerdBoy1392 cung cấp (Giấy phép Creative Commons BY-SA 3.0). Hình ảnh được sao chép từ <https://en.wikipedia.org/wiki/Tesseract>.
 3. Một sự thật thú vị: bất cứ ai bạn quen biết có lẽ đều là người cực đoan ở ít nhất một khía cạnh nào đó (ví dụ: mức độ cực đoan đến mức nào). (Lượng đường họ cho vào cà phê), nếu bạn xem xét đủ các khía cạnh.
 4. Karl Pearson, "Về các đường thẳng và mặt phẳng phù hợp nhất với các hệ điểm trong không gian," The Tạp chí Triết học và Khoa học Luân Đôn, Edinburgh và Dublin 2, số 11 (1901): 559-572, <https://hom1.info/pca>.
 5. Scikit-Learn sử dụng thuật toán được mô tả trong David A. Ross et al., "Incremental Learning for Robust Visual Tracking," International Journal of Computer Vision 77, no. 1-3 (2008): 125- 141.
 6. ϵ là chữ cái Hy Lạp epsilon, thường được dùng để chỉ các giá trị rất nhỏ.
 7. Sanjoy Dasgupta và cộng sự, "Một thuật toán mạng nơ-ron cho một vấn đề tính toán cơ bản," Science 358, số 6364 (2017): 793-796.
 8. Sam T. Roweis và Lawrence K. Saul, "Giảm chiều không tuyến tính bằng cách cực bộ "Nhúng tuyến tính," Khoa học 290, số 5500 (2000): 2323-2326.

OceanofPDE.com

Chương 9. Các kỹ thuật học không giám sát

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 9 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Mặc dù hầu hết các ứng dụng của Học máy hiện nay đều dựa trên học có giám sát (và do đó, đây là lĩnh vực mà hầu hết các khoản đầu tư đều tập trung vào), nhưng phần lớn dữ liệu hiện có lại không được gắn nhãn: chúng ta có các đặc trưng đầu vào X , nhưng không có nhãn y . Nhà khoa học máy tính Yann LeCun từng nổi tiếng nói rằng “nếu trí thông minh là một chiếc bánh, thì học không giám sát sẽ là chiếc bánh, học có giám sát sẽ là lớp kem phủ trên bánh, và học tăng cường sẽ là quả anh đào trên bánh”. Nói cách khác, tiềm năng của học không giám sát là rất lớn mà chúng ta mới chỉ bắt đầu khai thác.

Giả sử bạn muốn tạo một hệ thống chụp vài bức ảnh của từng sản phẩm trên dây chuyền sản xuất và phát hiện những sản phẩm bị lỗi.

Bạn hoàn toàn có thể tạo ra một hệ thống chụp ảnh tự động khá dễ dàng, và hệ thống này có thể cung cấp cho bạn hàng nghìn bức ảnh mỗi ngày. Sau đó, bạn có thể xây dựng một tập dữ liệu tương đối lớn chỉ trong vài tuần. Nhưng khoan đã, chưa có nhãn cho ảnh!

Nếu bạn muốn huấn luyện một bộ phân loại nhị phân thông thường để dự đoán xem một mặt hàng có bị lỗi hay không, bạn sẽ cần gắn nhãn cho từng bức ảnh.

Việc phân loại thành “hàng lỗi” hay “hàng bình thường” thường đòi hỏi các chuyên gia phải ngồi xuống và xem xét thủ công tất cả các hình ảnh. Đây là một công việc tốn thời gian, tốn kém và tẻ nhạt, vì vậy nó thường chỉ được thực hiện trên một tập hợp nhỏ các hình ảnh có sẵn. Kết quả là, tập dữ liệu được gắn nhãn sẽ khá nhỏ, và hiệu suất của thuật toán phân loại sẽ không như mong đợi. Hơn nữa, mỗi khi công ty thực hiện bất kỳ thay đổi nào đối với sản phẩm của mình, toàn bộ quy trình sẽ cần phải bắt đầu lại từ đầu. Sẽ thật tuyệt nếu thuật toán có thể tận dụng dữ liệu chưa được gắn nhãn mà không cần con người phải gắn nhãn cho từng hình ảnh?

Hãy cùng tìm hiểu về học không giám sát.

Trong **Chương 8**, chúng ta đã xem xét nhiệm vụ học không giám sát phổ biến nhất: giảm chiều dữ liệu. Trong chương này, chúng ta sẽ xem xét thêm một vài nhiệm vụ học không giám sát khác:

Phân cụm

Mục tiêu là nhóm các trường hợp tương tự lại với nhau thành các cụm. Phân cụm là một công cụ tuyệt vời cho phân tích dữ liệu, phân khúc khách hàng, hệ thống đề xuất, công cụ tìm kiếm, phân đoạn hình ảnh, học bán giám sát, giảm chiều dữ liệu, và nhiều hơn nữa.

Phát hiện bất thường (còn gọi là phát hiện ngoại lệ)

Mục tiêu là tìm hiểu dữ liệu “bình thường” trông như thế nào, sau đó sử dụng điều đó để phát hiện các trường hợp bất thường. Những trường hợp này được gọi là dị thường, hay ngoại lệ, trong khi các trường hợp bình thường được gọi là nội lệ. Phát hiện dị thường hữu ích trong nhiều ứng dụng khác nhau, chẳng hạn như phát hiện gian lận, phát hiện sản phẩm lỗi trong sản xuất, xác định xu hướng mới trong chuỗi thời gian hoặc loại bỏ các ngoại lệ khỏi tập dữ liệu trước khi huấn luyện một mô hình khác, điều này có thể cải thiện đáng kể hiệu suất của mô hình kết quả.

Ước tính mật độ

Đây là nhiệm vụ ước tính hàm mật độ xác suất (PDF) của quá trình ngẫu nhiên đã tạo ra tập dữ liệu. Ước tính mật độ thường được sử dụng để phát hiện bất thường: các trường hợp nằm ở mức rất thấp-

Các vùng có mật độ dân số cao thường là những điểm bất thường. Điều này cũng hữu ích cho việc phân tích và trực quan hóa dữ liệu.

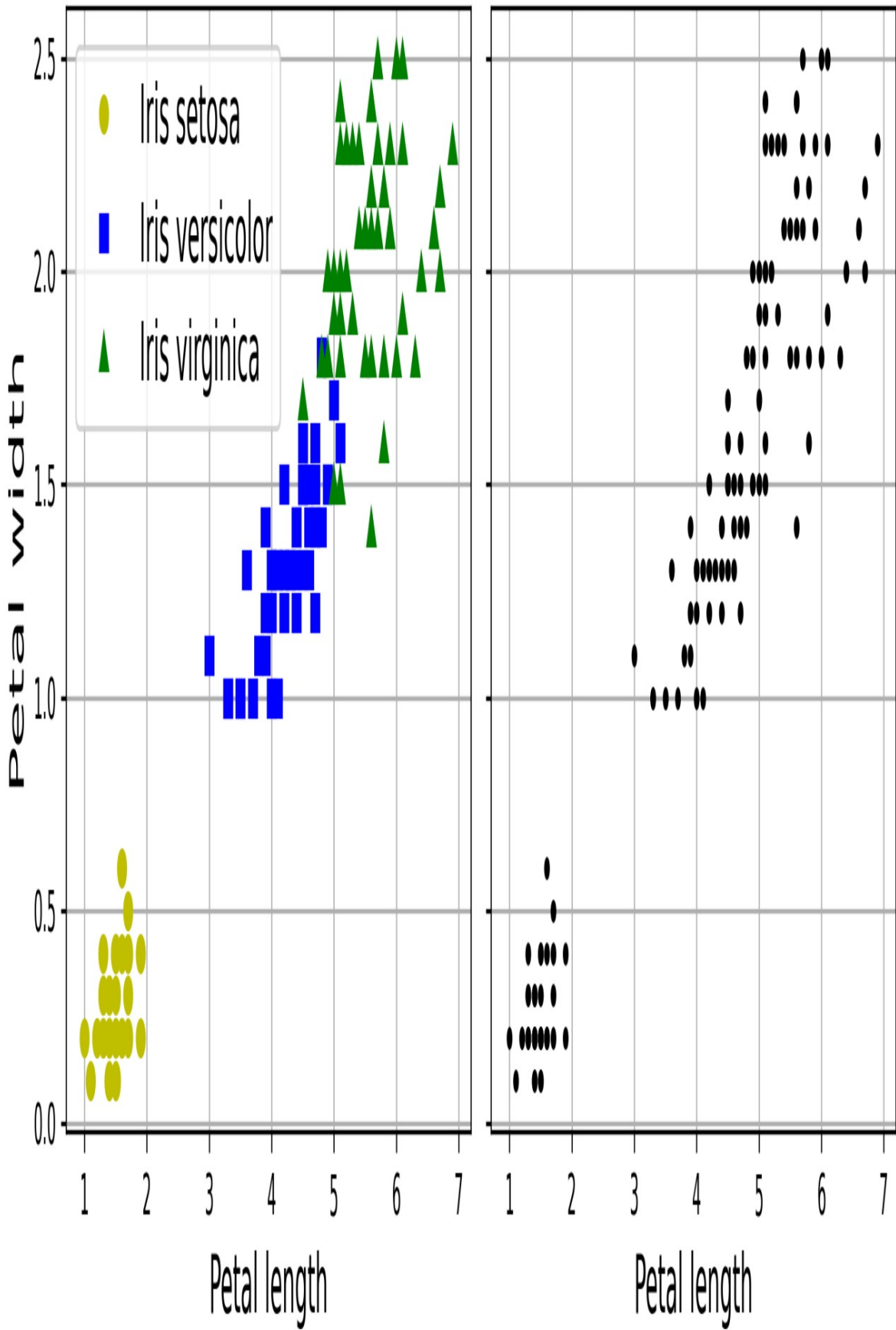
Sẵn sàng thưởng thức bánh ngọt chưa? Chúng ta sẽ bắt đầu với hai thuật toán phân cụm, K-Means và DBSCAN, sau đó chúng ta sẽ thảo luận về mô hình hỗn hợp Gaussian và xem cách chúng có thể được sử dụng để ước lượng mật độ, phân cụm và phát hiện bất thường.

Các thuật toán phân cụm: K-Means và DBSCAN

Trong một chuyến đi bộ đường dài trên núi, bạn tình cờ bắt gặp một loại cây chưa từng thấy trước đây. Bạn nhìn xung quanh và nhận thấy thêm một vài cây nữa. Chúng không hoàn toàn giống nhau, nhưng lại đủ tương đồng để bạn biết rằng chúng rất có thể thuộc cùng một loài (hoặc ít nhất là cùng một chi). Có thể bạn cần một nhà thực vật học để xác định loài đó là gì, nhưng chắc chắn bạn không cần chuyên gia để nhận diện các nhóm đối tượng trông giống nhau. Điều này được gọi là phân cụm: đó là nhiệm vụ xác định các trường hợp tương tự và gán chúng vào các cụm, hay các nhóm các trường hợp tương tự.

Cũng giống như trong phân loại, mỗi trường hợp được gán vào một nhóm. Tuy nhiên, không giống như phân loại, phân cụm là một nhiệm vụ không giám sát. Hãy xem **Hình 9-1**: bên trái là tập dữ liệu iris (được giới thiệu trong **Chương 4**), trong đó loài của mỗi trường hợp (tức là lớp của nó) được biểu thị bằng một dấu hiệu khác nhau. Đây là một tập dữ liệu được gán nhãn, rất phù hợp với các thuật toán phân loại như Hồi quy Logistic, SVM hoặc thuật toán Rừng ngẫu nhiên. Bên phải là cùng một tập dữ liệu, nhưng không có nhãn, vì vậy bạn không thể sử dụng thuật toán phân loại nữa. Đây là lúc các thuật toán phân cụm phát huy tác dụng: nhiều thuật toán có thể dễ dàng phát hiện cụm phía dưới bên trái. Cũng khá dễ dàng để nhận thấy bằng mắt thường, nhưng không rõ ràng lắm rằng cụm phía trên bên phải bao gồm hai cụm con riêng biệt. Tuy nhiên, tập dữ liệu có hai đặc điểm bổ sung (chiều dài và chiều rộng đài hoa), không được biểu thị ở đây, và các thuật toán phân cụm có thể tận dụng tốt tất cả các đặc điểm này, vì vậy trên thực tế chúng...

Xác định ba cụm khá tốt (ví dụ, sử dụng mô hình hỗn hợp Gaussian, chỉ có 5 trường hợp trong số 150 trường hợp được gán vào cụm sai).



Hình 9-1. Phân loại (bên trái) so với phân cụm (bên phải)

Phân cụm được sử dụng trong rất nhiều ứng dụng khác nhau, bao gồm:

Để phân khúc khách hàng

Bạn có thể phân nhóm khách hàng dựa trên các giao dịch mua hàng và hoạt động của họ trên trang web của bạn. Điều này rất hữu ích để hiểu khách hàng của bạn là ai và họ cần gì, từ đó bạn có thể điều chỉnh sản phẩm và các chiến dịch tiếp thị cho từng phân khúc. Ví dụ, phân khúc khách hàng có thể hữu ích trong các hệ thống đề xuất để gợi ý nội dung mà những người dùng khác trong cùng nhóm đã thích.

Để phân tích dữ liệu

Khi phân tích một tập dữ liệu mới, việc chạy thuật toán phân cụm và sau đó phân tích từng cụm riêng biệt có thể rất hữu ích.

Là một kỹ thuật giảm chiều dữ liệu

Sau khi dữ liệu được phân cụm, thông thường có thể đo lường mức độ tương đồng của mỗi trường hợp với từng cụm: mức độ tương đồng là bất kỳ thước đo nào về mức độ phù hợp của một trường hợp với một cụm. Vectơ đặc trưng x của mỗi trường hợp sau đó có thể được thay thế bằng vectơ thể hiện mức độ tương đồng của nó với các cụm. Nếu có k cụm, thì vectơ này có k chiều. Vectơ này thường có chiều thấp hơn nhiều so với vectơ đặc trưng ban đầu, nhưng nó có thể giữ lại đủ thông tin để xử lý tiếp.

Đối với kỹ thuật đặc trưng

Các đặc điểm tương đồng giữa các cụm thường hữu ích như những tính năng bổ sung. Ví dụ, trong [Chương 2](#), chúng tôi đã sử dụng thuật toán K-Means để thêm các đặc điểm tương đồng về địa lý giữa các cụm vào tập dữ liệu nhà ở California, và chúng đã giúp chúng tôi đạt được hiệu suất tốt hơn.

Dùng để phát hiện bất thường (còn gọi là phát hiện dữ liệu ngoại lai)

Bất kỳ trường hợp nào có độ liên kết thấp với tất cả các cụm đều có khả năng là một sự bất thường. Ví dụ, nếu bạn đã phân cụm người dùng của trang web của mình.

Dựa trên hành vi của họ, bạn có thể phát hiện những người dùng có hành vi bất thường, chẳng hạn như số lượng yêu cầu mỗi giây bất thường.

Đối với học bán giám sát

Nếu bạn chỉ có một vài nhãn, bạn có thể thực hiện phân cụm và truyền các nhãn đó đến tất cả các trường hợp trong cùng một cụm. Kỹ thuật này có thể làm tăng đáng kể số lượng nhãn có sẵn cho thuật toán học có giám sát tiếp theo, và do đó cải thiện hiệu suất của nó.

Dành cho các công cụ tìm kiếm

Một số công cụ tìm kiếm cho phép bạn tìm kiếm hình ảnh tương tự với hình ảnh tham chiếu. Để xây dựng hệ thống như vậy, trước tiên bạn sẽ áp dụng thuật toán phân cụm cho tất cả các hình ảnh trong cơ sở dữ liệu của mình; các hình ảnh tương tự sẽ được đưa vào cùng một cụm. Sau đó, khi người dùng cung cấp hình ảnh tham chiếu, tất cả những gì bạn cần làm là sử dụng mô hình phân cụm đã được huấn luyện để tìm cụm của hình ảnh này, và sau đó bạn chỉ cần trả về tất cả các hình ảnh từ cụm đó.

Để phân đoạn hình ảnh

Bằng cách nhóm các điểm ảnh theo màu sắc, sau đó thay thế màu của mỗi điểm ảnh bằng màu trung bình của nhóm đó, có thể giảm đáng kể số lượng màu sắc khác nhau trong ảnh. Phân đoạn ảnh được sử dụng trong nhiều hệ thống phát hiện và theo dõi đối tượng, vì nó giúp dễ dàng phát hiện đường viền của từng đối tượng.

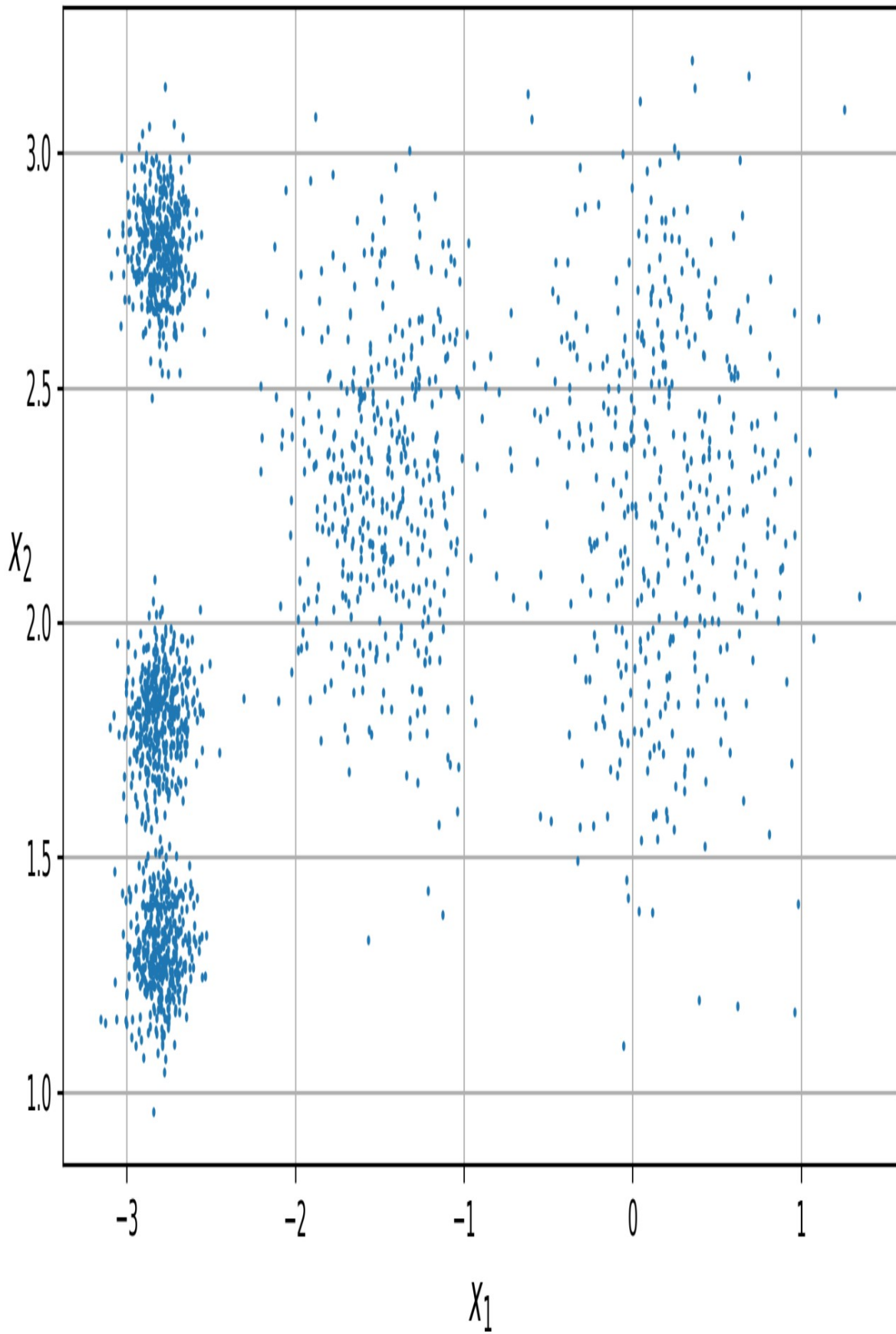
Không có định nghĩa chung nào về cụm dữ liệu: nó thực sự phụ thuộc vào ngữ cảnh, và các thuật toán khác nhau sẽ nắm bắt được các loại cụm dữ liệu khác nhau.

Một số thuật toán tìm kiếm các trường hợp tập trung xung quanh một điểm cụ thể, được gọi là trọng tâm. Những thuật toán khác tìm kiếm các vùng liên tục gồm các trường hợp được phân bố dày đặc: các cụm này có thể có bất kỳ hình dạng nào. Một số thuật toán có tính phân cấp, tìm kiếm các cụm con của các cụm. Và danh sách còn dài nữa.

Trong phần này, chúng ta sẽ xem xét hai thuật toán phân cụm phổ biến là K-Means và DBSCAN, và khám phá một số ứng dụng của chúng, chẳng hạn như giảm chiều dữ liệu phi tuyến tính, học bán giám sát và phát hiện bất thường.

K-Means

Hãy xem xét tập dữ liệu chưa được gán nhãn được thể hiện trong **Hình 9-2**: bạn có thể thấy rõ năm nhóm dữ liệu. Thuật toán K-Means là một thuật toán đơn giản có khả năng phân cụm loại dữ liệu này rất nhanh chóng và hiệu quả, thường chỉ trong vài lần lặp. Nó được Stuart Lloyd đề xuất tại Bell Labs vào năm 1957 như một kỹ thuật điều chế mã xung, nhưng mãi đến năm 1982 nó mới được công bố bên ngoài công ty¹. Năm 1965, Edward W. Forgy đã công bố một thuật toán gần như tương tự, vì vậy K-Means đôi khi được gọi là Lloyd-Forgy.



Hình 9-2. Một tập dữ liệu chưa được gán nhãn bao gồm năm nhóm dữ liệu (blob) các trường hợp.

Hãy huấn luyện một thuật toán phân cụm K-Means trên tập dữ liệu này. Thuật toán sẽ cố gắng tìm tâm của mỗi vùng dữ liệu (blob) và gán mỗi trường hợp vào vùng dữ liệu gần nhất:

```
from sklearn.cluster import KMeans from
sklearn.datasets import make_blobs

X, y = make_blobs([...]) # tạo các blob: y chứa ID cụm, nhưng chúng ta

# Chúng tôi sẽ không sử dụng chúng, đó là điều chúng tôi muốn

để dự đoán k = 5
kmeans
= KMeans(n_clusters=k, random_state=42) y_pred = kmeans.fit_predict(X)
```

Lưu ý rằng bạn phải chỉ định số lượng cụm k mà thuật toán phải tìm. Trong ví dụ này, nhìn vào dữ liệu thì khá rõ ràng là k nên được đặt là 5, nhưng nói chung thì không dễ như vậy. Chúng ta sẽ thảo luận về điều này ngay sau đây.

Mỗi trường hợp được gán vào một trong năm cụm. Trong ngữ cảnh phân cụm, nhãn của một trường hợp là chỉ số của cụm mà thuật toán gán cho trường hợp đó: điều này không nên nhầm lẫn với nhãn lớp trong phân loại, được sử dụng làm mục tiêu (hãy nhớ rằng phân cụm là một nhiệm vụ học không giám sát). Trường hợp KMeans lưu giữ các nhãn dự đoán của các trường hợp mà nó được huấn luyện, có sẵn thông qua biến trường hợp `labels_`:

```
>>> y_pred
array([4, 0, 1, ..., 2, 1, 0], dtype=int32) >>> y_pred is
kmeans.labels_ True
```

Chúng ta cũng có thể xem xét năm tâm cụm mà thuật toán đã tìm thấy:

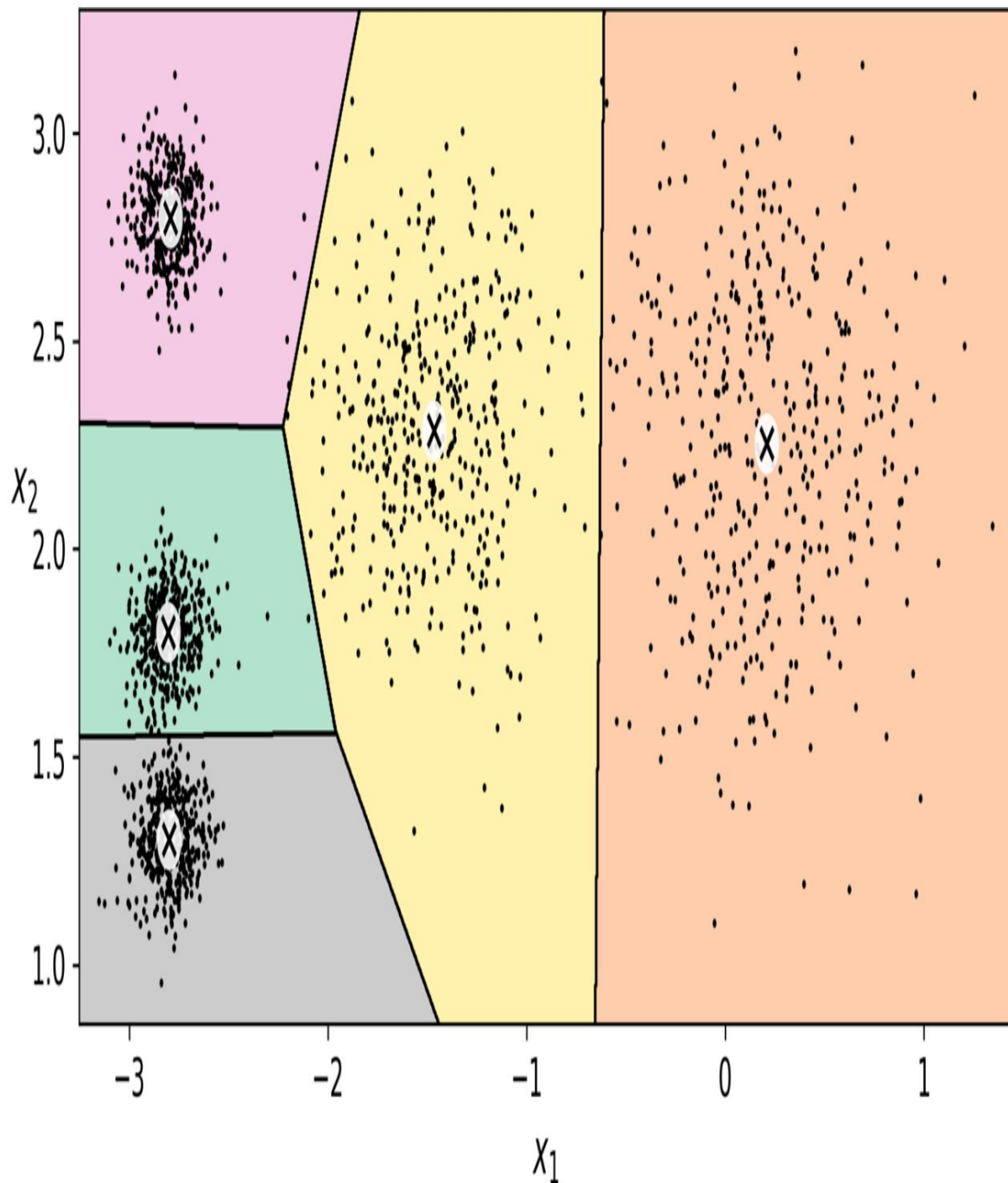
```
>>> kmeans.cluster_centers_
array([[ -2.80389616,  1.80117999], [ 0.20876306,
      2.25551336], [ -2.79290307,  2.79641063],
```

```
[-1.46679593, 2.28585348],  
[-2.80037642, 1.30082566]])
```

Bạn có thể dễ dàng gán các phiên bản mới vào cụm có tâm cụm gần nhất:

```
>>> import numpy as np  
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]]) >>>  
kmeans.predict(X_new)  
array([1, 1, 2, 2], dtype=int32)
```

Nếu bạn vẽ các ranh giới quyết định của cụm, bạn sẽ nhận được một phép phân vùng Voronoi: xem [Hình 9-3](#), trong đó mỗi tâm cụm được biểu thị bằng một chữ X.



Hình 9-3. Ranh giới quyết định K-Means (phân vùng Voronoi)

Đa số các trường hợp được phân loại rõ ràng vào cụm thích hợp, nhưng một vài trường hợp có thể bị dán nhãn sai, đặc biệt là gần ranh giới giữa cụm trên cùng bên trái và cụm trung tâm. Thật vậy, thuật toán K-Means hoạt động không tốt lắm khi các khối dữ liệu có kích thước rất nhỏ.

các đường kính khác nhau vì khi gán một instance vào một cụm, nó chỉ quan tâm đến khoảng cách đến tâm cụm.

Thay vì gán mỗi trường hợp vào một cụm duy nhất, được gọi là phân cụm cứng, việc gán cho mỗi trường hợp một điểm số cho mỗi cụm có thể hữu ích hơn, được gọi là phân cụm mềm. Điểm số có thể là khoảng cách giữa trường hợp đó và tâm cụm; ngược lại, nó có thể là điểm số tương đồng (hoặc độ tương quan), chẳng hạn như hàm cơ sở xuyên tâm Gauss mà chúng ta đã sử dụng trong [Chương 2](#). Trong lớp KMeans, phương thức transform() đo khoảng cách từ mỗi trường hợp đến mọi tâm cụm:

```
>>> kmeans.transform(X_new).round(2)
array([[2.81, 0.33, 2.9 , 1.49, 2.89],
       [5.81, 2.8 , 5.85, 4.48, 5.84],
       [1.21, 3.29, 0.29, 1.69, 1.71],
       [0.73, 3.22, 0.36, 1.55, 1.22]])
```

Trong ví dụ này, phần tử đầu tiên trong X_new nằm cách tâm cụm thứ nhất khoảng 2,81, cách tâm cụm thứ hai 0,33, cách tâm cụm thứ ba 2,90, cách tâm cụm thứ tư 1,49 và cách tâm cụm thứ năm 2,89. Nếu bạn có một tập dữ liệu đa chiều và bạn biến đổi nó theo cách này, bạn sẽ thu được một tập dữ liệu k chiều: phép biến đổi này có thể là một kỹ thuật giảm chiều phi tuyến tính rất hiệu quả. Ngoài ra, bạn có thể sử dụng các khoảng cách này làm các đặc trưng bổ sung để huấn luyện một mô hình khác, như chúng ta đã làm trong [Chương 2](#).

Thuật toán K-Means Vậy thuật

toán này hoạt động như thế nào? Giả sử bạn được cung cấp các tâm cụm. Bạn có thể dễ dàng gán nhãn cho tất cả các trường hợp trong tập dữ liệu bằng cách gán mỗi trường hợp vào cụm có tâm cụm gần nhất. Ngược lại, nếu bạn được cung cấp tất cả các nhãn của các trường hợp, bạn có thể dễ dàng xác định vị trí tâm cụm của mỗi cụm bằng cách tính trung bình các trường hợp trong cụm đó. Nhưng bạn không được cung cấp cả nhãn lẫn tâm cụm, vậy bạn có thể tiến hành như thế nào? Hãy bắt đầu bằng cách đặt các tâm cụm một cách ngẫu nhiên (ví dụ: bằng cách chọn ngẫu nhiên k trường hợp từ tập dữ liệu và sử dụng vị trí của chúng làm tâm cụm). Sau đó, gán nhãn cho các trường hợp, cập nhật tâm cụm, gán nhãn cho các trường hợp, cập nhật tâm cụm,

và cứ thế tiếp tục cho đến khi các tâm cụm ngừng di chuyển. Thuật toán được đảm bảo hội tụ trong một số bước hữu hạn (thường khá nhỏ). Điều đó là bởi vì khoảng cách bình phương trung bình giữa các cụm và tâm cụm gần nhất của chúng chỉ có thể giảm xuống ở mỗi bước, và vì nó không thể âm, nên nó được đảm bảo hội tụ.

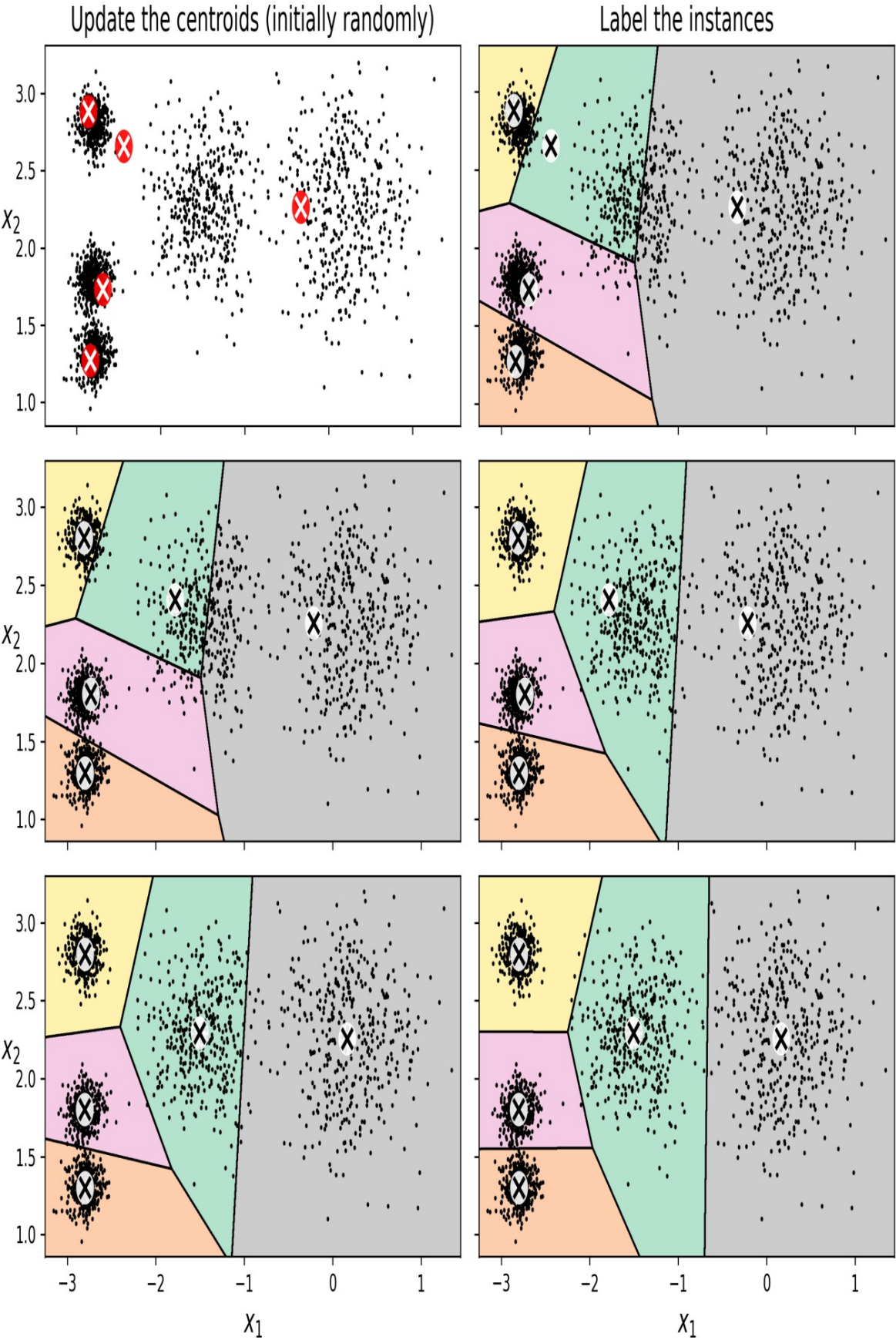
Bạn có thể thấy thuật toán hoạt động trong **Hình 9-4**: các tâm cụm được khởi tạo ngẫu nhiên (trên cùng bên trái), sau đó các trường hợp được gán nhãn (trên cùng bên phải), tiếp theo các tâm cụm được cập nhật (giữa bên trái), các trường hợp được gán nhãn lại (giữa bên phải), và cứ thế tiếp tục. Như bạn thấy, chỉ sau ba lần lặp, thuật toán đã đạt được kết quả phân cụm gần như tối ưu.

GHI CHÚ

Độ phức tạp tính toán của thuật toán nhìn chung là tuyến tính đối với số lượng trường hợp m , số lượng cụm k và số chiều n .

Tuy nhiên, điều này chỉ đúng khi dữ liệu có cấu trúc phân cụm. Nếu không, trong trường hợp xấu nhất, độ phức tạp có thể tăng theo cấp số nhân với số lượng mẫu.

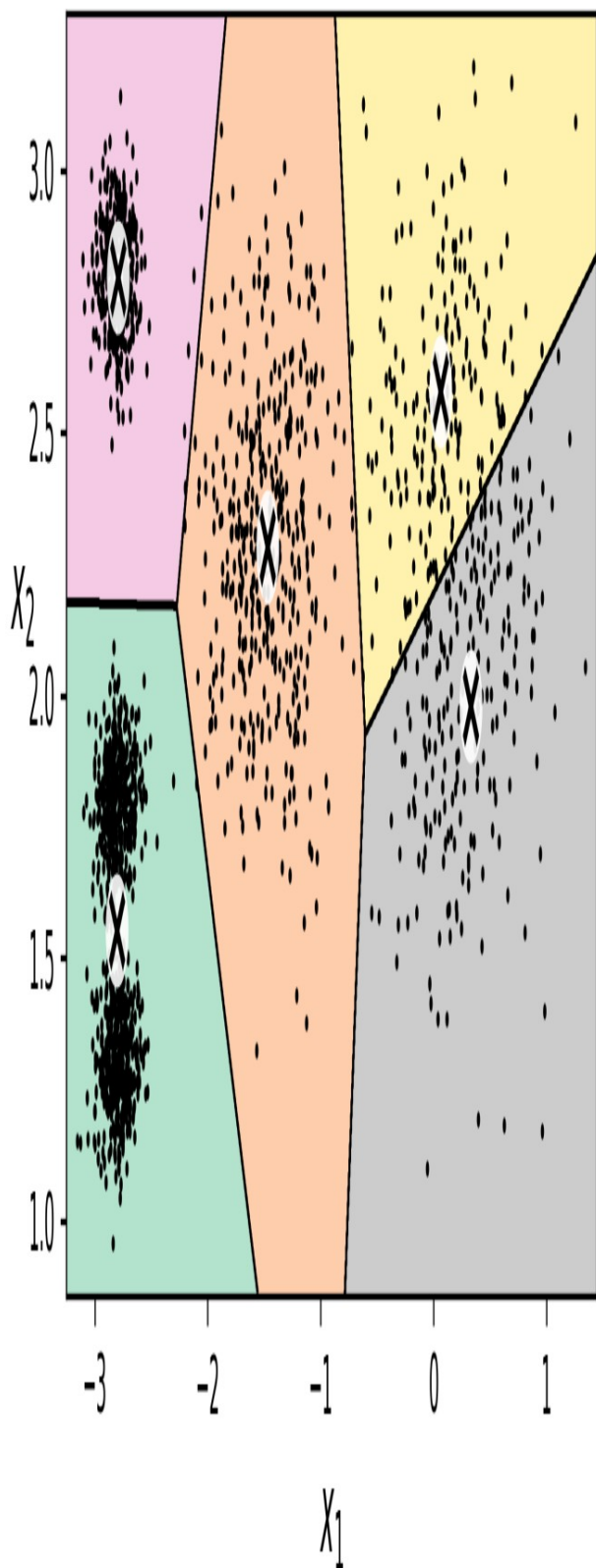
Trên thực tế, điều này hiếm khi xảy ra, và K-Means thường là một trong những thuật toán phân cụm nhanh nhất.



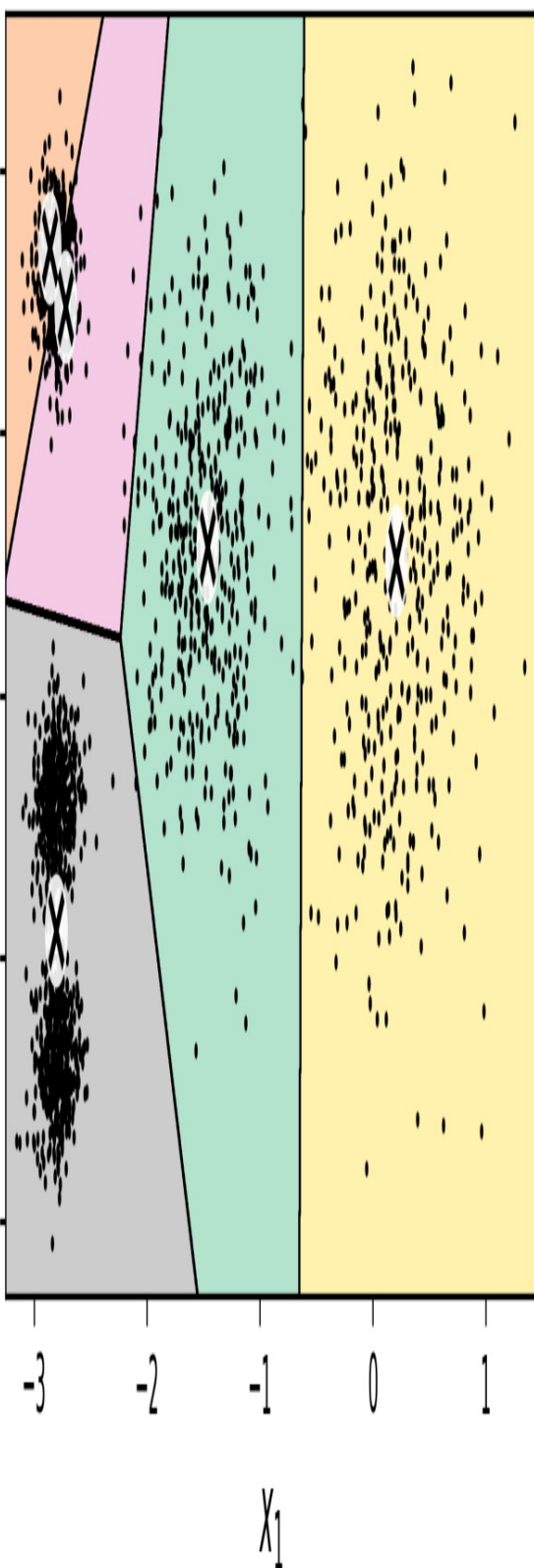
Hình 9-4. Thuật toán K-Means

Mặc dù thuật toán được đảm bảo sẽ hội tụ, nhưng nó có thể không hội tụ đến giải pháp đúng (tức là, nó có thể hội tụ đến một điểm tối ưu cục bộ): việc này phụ thuộc vào việc khởi tạo trọng tâm. Hình 9-5 cho thấy hai giải pháp không tối ưu mà thuật toán có thể hội tụ đến nếu bạn không may mắn với bước khởi tạo ngẫu nhiên.

Solution 1



Solution 2 (with a different random init)



Hình 9-5. Các giải pháp không tối ưu do khởi tạo tâm trọng lực không may mắn.

Hãy cùng xem xét một vài cách bạn có thể giảm thiểu rủi ro này bằng cách cải thiện quá trình khởi tạo trọng tâm.

Phương pháp khởi tạo tâm

Nếu bạn biết vị trí gần đúng của các tâm cụm (ví dụ: nếu bạn đã chạy một thuật toán phân cụm khác trước đó), thì bạn có thể đặt siêu tham số `init` thành một mảng NumPy chứa danh sách các tâm cụm và đặt `n_init` thành 1:

```
good_init = np.array([[ -3, 3], [ -3, 2], [ -3, 1], [ -1, 2], [ 0, 2]]) kmeans =
KMeans(n_clusters=5, init=good_init, n_init=1, random_state=42)
kmeans.fit(X)
```

Một giải pháp khác là chạy thuật toán nhiều lần với các giá trị khởi tạo ngẫu nhiên khác nhau và giữ lại giải pháp tốt nhất. Số lượng giá trị khởi tạo ngẫu nhiên được điều khiển bởi siêu tham số `n_init`: theo mặc định, nó bằng 10, có nghĩa là toàn bộ thuật toán được mô tả ở trên sẽ chạy 10 lần khi bạn gọi hàm `fit()`, và Scikit-Learn sẽ giữ lại giải pháp tốt nhất. Nhưng chính xác thì nó biết giải pháp nào là tốt nhất bằng cách nào? Nó sử dụng một chỉ số hiệu suất! Chỉ số đó được gọi là quán tính của mô hình, là tổng bình phương khoảng cách giữa các trường hợp và tâm gần nhất của chúng. Nó xấp xỉ bằng 219,4 đối với mô hình bên trái trong Hình 9-5, 258,6 đối với mô hình bên phải trong Hình 9-5 và chỉ 211,6 đối với mô hình trong Hình 9-3. Lớp KMeans chạy thuật toán `n_init` lần và giữ lại mô hình có quán tính thấp nhất. Trong ví dụ này, mô hình trong Hình 9-3 sẽ được chọn (trừ khi chúng ta rất không may mắn với `n_init` lần khởi tạo ngẫu nhiên liên tiếp). Nếu bạn tò mò, quán tính của một mô hình có thể được truy cập thông qua biến thể hiện `inertia_`:

```
>>> kmeans.inertia_
211.59853725816836
```

Phương thức `score()` trả về quán tính âm. Tại sao lại là âm? Bởi vì phương thức `score()` của một mô hình dự đoán phải luôn tuân thủ quy tắc "càng lớn càng tốt" của Scikit-Learn: nếu một mô hình dự đoán tốt hơn mô hình khác, phương thức `score()` của nó sẽ trả về điểm số cao hơn.

```
>>> kmeans.score(X)
-211.5985372581684
```

Một cải tiến quan trọng cho thuật toán K-Means, K-Means++, đã được đề xuất trong một [bài báo năm 2006](#)². Do David Arthur và Sergei Vassilvitskii đề xuất. Họ đã giới thiệu một bước khởi tạo thông minh hơn, có xu hướng chọn các tâm cụm cách xa nhau, và cải tiến này làm cho thuật toán K-Means ít có khả năng hội tụ đến một giải pháp không tối ưu. Họ đã chứng minh rằng việc tính toán bổ sung cần thiết cho bước khởi tạo thông minh hơn là rất đáng giá vì nó cho phép giảm đáng kể số lần thuật toán cần được chạy để tìm ra giải pháp tối ưu. Thuật toán khởi tạo K-Means++ hoạt động như sau:

1. Chọn một trọng tâm $c^{(1)}$, được chọn ngẫu nhiên đồng đều từ tập dữ liệu.

2. Lấy một tâm mới c , chọn một trường hợp x với xác suất $D(x)^{(T\acute{o}i)}$

$$D(x^{(i)})^2 / \sum_{j=1}^m D(x^{(j)})^2$$
, (i) trong đó $D(x)$ là khoảng cách giữa $x^{(i)}$ và tâm gần nhất đã được chọn. Phân bố xác suất này đảm bảo rằng các trường hợp càng xa các tâm đã được chọn thì càng có nhiều khả năng được chọn làm tâm.

3. Lặp lại bước trước đó cho đến khi tất cả k tâm cụm đã được chọn.

Lớp KMeans sử dụng phương thức khởi tạo này theo mặc định.

K-Means tăng tốc và K-Means theo lô nhỏ

Một cải tiến khác cho thuật toán K-Means đã được đề xuất trong một [bài báo năm 2003](#)³ của Charles Elkan. Trên một số tập dữ liệu lớn với nhiều cụm, thuật toán có thể được tăng tốc bằng cách tránh nhiều phép tính khoảng cách không cần thiết. Elkan đã đạt được điều này bằng cách khai thác bất đẳng thức tam giác (tức là,

Thuật toán Elkan dựa trên nguyên tắc đường thẳng luôn là khoảng cách ngắn nhất giữa hai điểm, bằng cách theo dõi giới hạn dưới và giới hạn trên của khoảng cách giữa các điểm dữ liệu và tâm cụm. Tuy nhiên, thuật toán Elkan không phải lúc nào cũng tăng tốc quá trình huấn luyện, nó phụ thuộc vào tập dữ liệu, và đôi khi thậm chí có thể làm chậm quá trình huấn luyện đáng kể. Dù vậy, nếu bạn muốn thử, hãy đặt `algorithm="elkan"`.

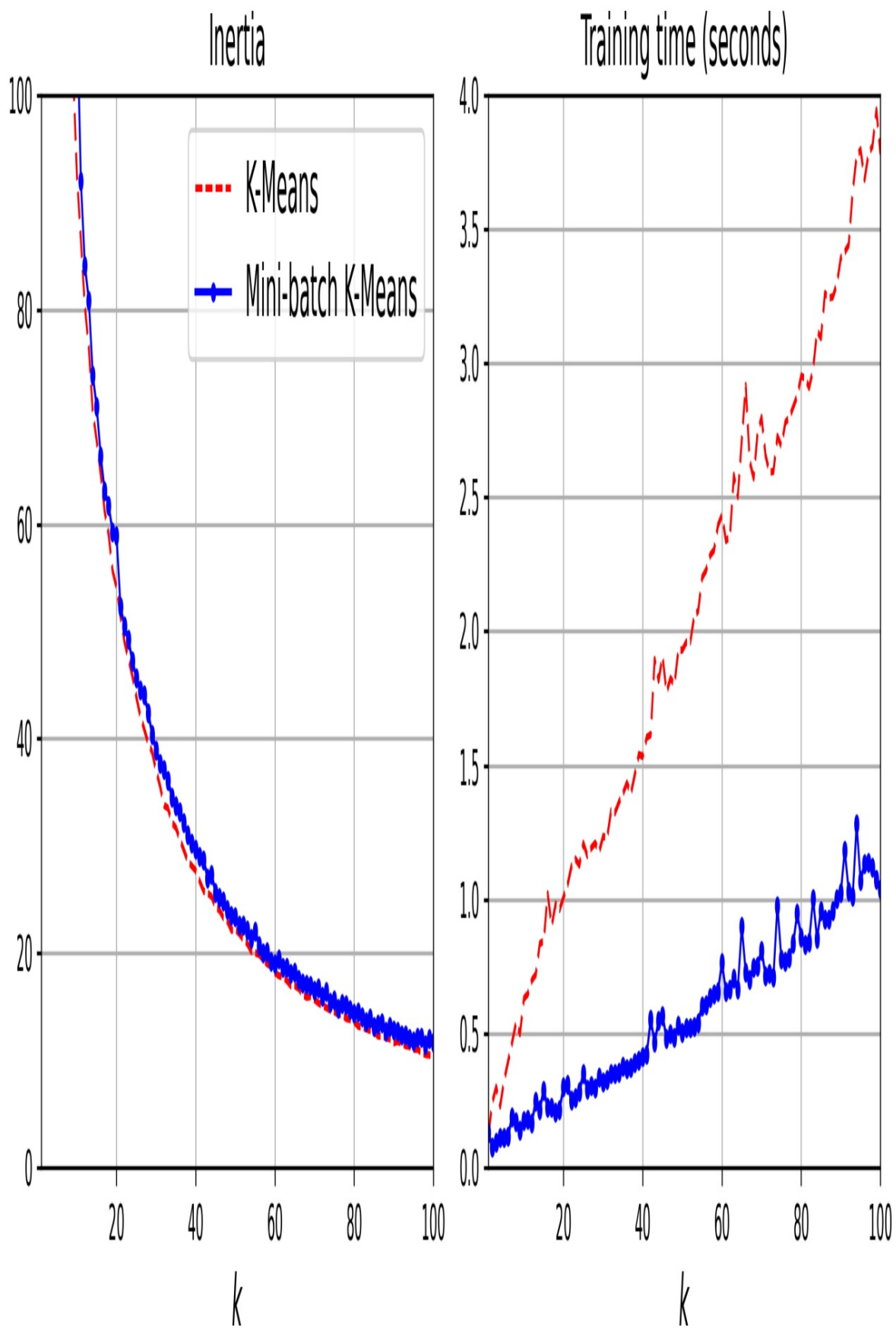
Một biến thể quan trọng khác của thuật toán K-Means đã được đề xuất trong một bài báo năm 2010. Thuật toán này được phát triển bởi David Sculley. Thay vì sử dụng toàn bộ tập dữ liệu ở mỗi lần lặp, thuật toán có khả năng sử dụng các mini-batch, chỉ di chuyển các tâm cụm một chút ở mỗi lần lặp. Điều này giúp tăng tốc thuật toán lên từ ba đến bốn lần và cho phép phân cụm các tập dữ liệu khổng lồ không vừa trong bộ nhớ. Scikit-Learn triển khai thuật toán này trong lớp `MiniBatchKMeans`. Bạn chỉ cần sử dụng lớp này giống như lớp `KMeans`:

```
from sklearn.cluster import MiniBatchKMeans

minibatch_kmeans = MiniBatchKMeans(n_clusters=5, random_state=42)
minibatch_kmeans.fit(X)
```

Nếu tập dữ liệu không vừa với bộ nhớ, lựa chọn đơn giản nhất là sử dụng lớp `mmap`, như chúng ta đã làm với PCA tăng dần trong Chương 8. Ngoài ra, bạn có thể truyền từng mini-batch một vào phương thức `partial_fit()`, nhưng điều này sẽ đòi hỏi nhiều công sức hơn, vì bạn sẽ cần thực hiện nhiều lần khởi tạo và tự chọn kết quả tốt nhất.

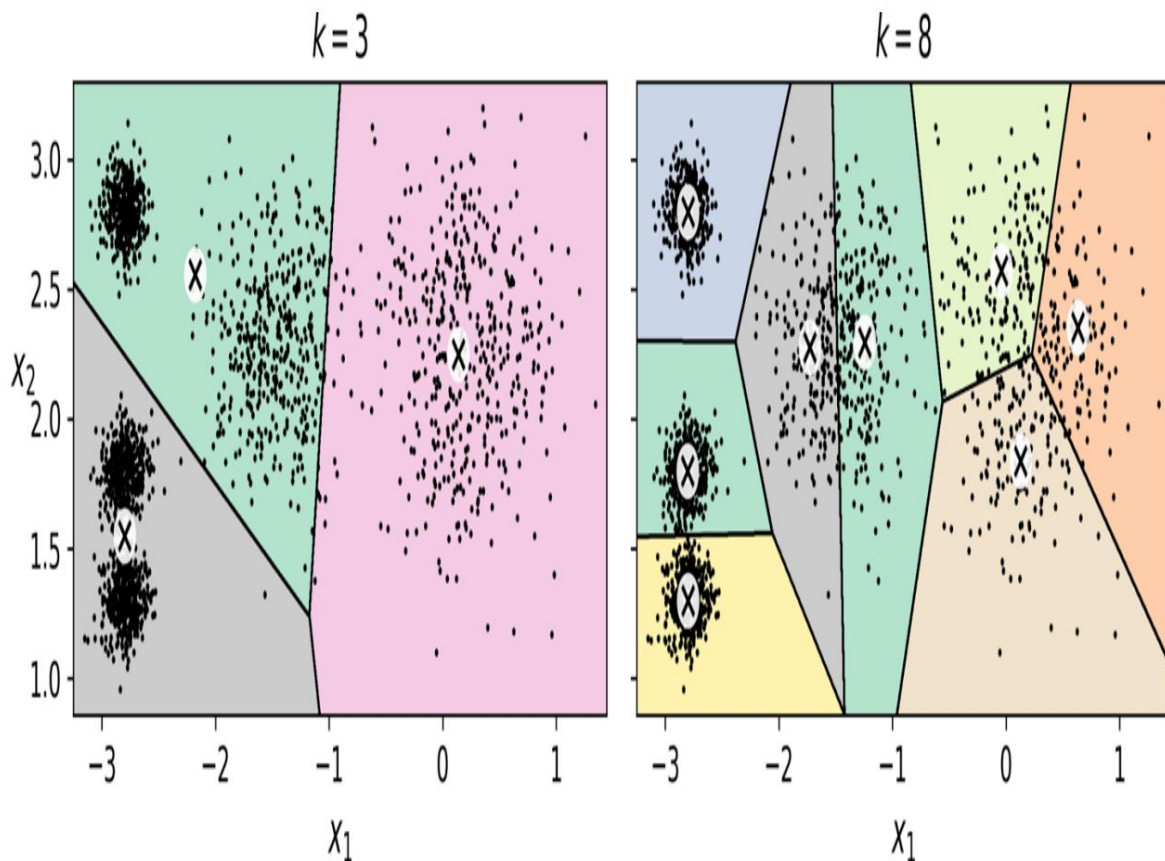
Mặc dù thuật toán Mini-batch K-Means nhanh hơn nhiều so với thuật toán K-Means thông thường, nhưng quán tính của nó nhìn chung lại kém hơn một chút. Bạn có thể thấy điều này trong Hình 9-6: biểu đồ bên trái so sánh quán tính của các mô hình Mini-batch K-Means và K-Means thông thường được huấn luyện trên tập dữ liệu năm khối trước đó bằng cách sử dụng các số lượng cụm k khác nhau. Sự khác biệt giữa hai đường cong nhỏ nhưng có thể nhìn thấy. Trong biểu đồ bên phải, bạn có thể thấy rằng Mini-batch K-Means nhanh hơn khoảng 3,5 lần so với K-Means thông thường trên tập dữ liệu này.



Hình 9-6. Thuật toán K-Means xử lý theo lô nhỏ có quán tính cao hơn thuật toán K-Means thông thường (bên trái) nhưng tốc độ nhanh hơn nhiều (bên phải), đặc biệt khi k tăng lên.

Tìm số lượng cụm tối ưu

Cho đến nay, chúng ta đã đặt số lượng cụm k là 5 vì nhìn vào dữ liệu, rõ ràng đây là số lượng cụm chính xác. Nhưng nhìn chung, việc xác định giá trị k không hề dễ dàng, và kết quả có thể khá tệ nếu bạn đặt sai giá trị. Như bạn có thể thấy trong **Hình 9-7**, việc đặt k là 3 hoặc 8 dẫn đến các mô hình khá kém.



Hình 9-7. Những lựa chọn sai lầm về số lượng cụm: khi k quá nhỏ, các cụm riêng biệt bị hợp nhất (bên trái), và khi k quá lớn, một số cụm bị chia thành nhiều phần (bên phải).

Có lẽ bạn đang nghĩ rằng chúng ta chỉ cần chọn mô hình có quán tính thấp nhất, phải không? Thật không may, mọi chuyện không đơn giản như vậy. Quán tính của $k=3$ là khoảng 653,2, cao hơn nhiều so với $k=5$ (211,6). Nhưng với $k=8$, quán tính chỉ là 119,1. Quán tính không phải là một chỉ số hiệu suất tốt khi cố gắng chọn k vì nó tiếp tục giảm khi chúng ta...